OFFICE OF THE CHIEF SYSTEMS ARCHITECT

TECHNICAL ARCHITECTURE SPECIFICATION

Opus Publica

RC2 Technical Architecture Specification

From Governance Document to Operating System

Version 1.0

Issued 09 August 2026

Document Type	Technical Documentation (Implementation Specification)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Architects, principal engineers, platform engineers building RC2
Classification	Normative — RC2 Architecture of Record
Builds On	RC2 Evolution Roadmap v1.0 + RC1 Constitution Vol I/II + Playbook
Operationalizes	RC2 Evolution Roadmap Chapter 7 (Implementation Architecture)
Effective Date	09 August 2026
Supersedes	Roadmap Chapter 7 (high-level); this spec is the detailed successor
Review Cadence	Per-RC milestone; quarterly architecture review

	thereafter
Document ID	OP-RC2-ARCH-001

This specification is the implementation blueprint for RC2. It converts the RC2 Evolution Roadmap into a deployable architecture: components, interfaces, data flow, deployment topology, security, observability, and limitations.

Chapter 1 — Introduction

1.1 Opening Hook: From Governance Document to Operating System

Opus Publica's RC1 Engineering Constitution is, at 1,285 pages across five documents, a complete statement of the platform's provisions. It is also, as the RC2 Evolution Roadmap observed, a document at the limit of human enforcement. The RC2 Roadmap answered the question “how do we make the Constitution machine-enforced?” by specifying five pillars — the Machine-Readable Constitution, the Automated Traceability Graph, the Continuous Certification Engine, the Governance Dashboard, and the Self-Auditing Platform — and naming a technology stack (OPA, Neo4j, Backstage, Grafana, Sigstore, OSCAL, OpenTelemetry, GitHub Actions, Flux) in which those pillars would be realized. The Roadmap is the strategic document. This specification is the implementation document. It answers the question that follows from the Roadmap: how, exactly, do we deploy those technologies, wire them together, and operate them as a single self-enforcing governance system?

The thesis of this specification is that the RC2 architecture is the implementation of a transition — a transition from a governance document that humans read to an operating system that machines execute. An operating system, in the classical sense, is a layer of software that manages resources, enforces policies, and provides services to applications. The RC2 architecture is, in that sense, an operating system for governance: it manages the resources of the engineering organization (repositories, services, evidence, certification records), enforces the policies of the Constitution (as OPA/Rego policies evaluated at every commit, deploy, and runtime decision), and provides services to the platform's applications (policy decisions on demand, traceability queries, certification status, audit reports). An engineer who, in RC1, opened a pull request and asked “does this satisfy the Constitution?” received an answer from a human reviewer who read the document. In RC2, the same engineer receives the answer from a machine that executes the document. The Constitution has not changed; what has changed is that the Constitution is now an executable program.

The RC2 Architecture Prime Directive

The RC2 architecture SHALL be implemented such that every constitutional provision that can be evaluated by a machine IS evaluated by a machine, at the earliest possible point in the engineering lifecycle, with the result recorded as evidence in the Automated Traceability Graph and reported as an OSCAL Assessment Result. Every component specified in this document SHALL be deployed with failure modes that fail safe (block rather than permit) and runtime verification that detects component failure within the SLO-defined window. There SHALL be no governance decision in RC2 that depends on a human reading a document to verify a fact a machine can check.

1.2 Purpose

This specification provides architects, principal engineers, and platform engineers with the information required to build, deploy, and operate the RC2 architecture. It specifies the components (Chapter 5), their data flow (Chapter 6), their deployment topology (Chapter 7), their security controls (Chapter 8), their observability (Chapter 9), and their limitations (Chapter 10). It is normative: where this specification uses RFC 2119 vocabulary (MUST, SHALL, SHOULD), the requirement is binding on RC2 implementation. Where the specification describes alternatives (e.g., Neo4j or Amazon Neptune), the choice is delegated to the Architecture Decision Records (ADRs) that this specification triggers.

1.3 Audience and Prerequisites

The intended audience is the RC2 implementation team: architects who make technology choices, principal engineers who design the components, and platform engineers who deploy and operate them. The reader is expected to be familiar with the RC1 Engineering Constitution (Volumes I and II), the RC2 Evolution Roadmap, and the technology stack named in Chapter 7 of the Roadmap. Familiarity with Kubernetes, GitOps (Flux or Argo CD), Open Policy Agent and Rego, Neo4j and Cypher, OpenTelemetry, Prometheus, Grafana, and Sigstore is assumed; this specification does not tutor those technologies, it specifies their use in RC2. Where a less-common technology is invoked (OSCAL, Conftest, CycloneDX, SLSA), the specification provides a brief definition and a reference.

1.4 Document Scope

This specification covers the architecture of RC2 — the components, their interfaces, their data flow, their deployment topology, their security and observability controls, and their limitations. It does not cover the RC1 constitutional provisions themselves (those are in the Constitution); the RC2 Roadmap's strategy and migration path (those are in the Roadmap); or the RC2 product features (multi-tenancy, plugin SDK, AI assistance — those are in separate RC2 product documents). The boundary is sharp: this specification is about how to build the operating system; it is not about the policy the operating system enforces, the strategy that motivated it, or the applications that run on it.

- **In scope:** the components of the RC2 control plane (governance) and data plane (application).
- **In scope:** the interfaces between components and the data flow through the system.
- **In scope:** the deployment topology (Kubernetes namespaces, environments, GitOps model).
- **In scope:** the security architecture (threat model, supply chain, audit trail).
- **In scope:** the observability architecture (metrics, logs, traces, dashboards, alerting).
- **Out of scope:** the constitutional provisions themselves — those are in the Constitution and the MRC.
- **Out of scope:** the migration path from RC1 to RC2 — that is in the Roadmap.
- **Out of scope:** RC2 product features (multi-tenancy, plugins, AI) — those are in RC2 product documents.

1.5 Definitions

The following architectural terms are used throughout this specification. The definitions are normative for this document; where a term is also defined in the Constitution or the Roadmap, the definitions are consistent.

Term	Definition
Machine-Readable Constitution (MRC)	A versioned YAML/JSON encoding of every constitutional provision, owned in Git, that is the single source of truth from which every human-readable document is generated.
Automated Traceability Graph (ATG)	A live property graph (Neo4j) whose nodes are engineering artifacts and whose edges are traceability relationships, built automatically from the codebase, the MRC, and CI metadata.
Continuous Certification Engine (CCE)	The integrated set of OPA policies, Conftest gates, GitHub Actions jobs, and OSCAL emitters that evaluate every certification criterion on every commit, deploy, and release.
Governance Dashboard	The Grafana (operator-facing) and Backstage (developer-facing) surfaces that report constitutional health, SLO burn, certification coverage, and audit findings in real time.
Self-Auditing Platform (SAP)	The nightly + continuous audit engine, with rule-based and ML-based anomaly detection, that realizes continuous assurance per NIST SP 800-137 ISCM.
Pillar	One of the five RC2 architectural pillars (MRC, ATG, CCE, Dashboard, SAP).
Control Plane	The set of RC2 components that govern the platform (MRC, OPA, Neo4j, GitHub Actions, Grafana, Backstage, Self-Audit). The control plane does not touch user-visible data; it enforces policy on the data plane.
Data Plane	The set of RC2 components that produce and consume user-visible data (the 20 application subsystems, the event backbone, storage, external integrations). The data plane is governed by the control plane.
Policy-as-Code	The discipline of expressing governance policies as versioned, testable, reviewable code (Rego), enforced at deploy and runtime. NIST SP 800-204C.
GitOps	The operational model in which Git is the source of

	truth for system state and a controller (Flux) reconciles live state to declared state. CNCF OpenGitOps.
Shift-Left	The principle of moving verification as early as possible in the engineering lifecycle (PR time vs. release time), reducing the cost of defect detection.
Continuous Assurance	Compliance verified continuously (per-commit, per-deploy, per-event) by automated tooling, rather than periodically by human auditors. ISACA framing.
OSCAL	Open Security Controls Assessment Language — a NIST-led machine-readable format for security control documentation, including Assessment Results.
SLSA	Supply-chain Levels for Software Artifacts — an OpenSSF framework defining supply-chain integrity levels (L1-L4) and provenance attestations.

1.6 References

This specification depends on the following documents and standards. The full reference list, with verified URLs, is in Appendix D.

- **RC2-ROAD:** RC2 Evolution Roadmap v1.0 — the strategic document this spec operationalizes.
- **RC1-CONST:** RC1 Engineering Constitution Vol I (25 chapters) and Vol II (11 chapters) — the provisions the architecture enforces.
- **RC1-PLAY:** RC1 Engineering Playbook — the operational procedures the architecture automates.
- **RFC-2119:** RFC 2119 — key words for use in RFCs to indicate requirement levels.
- **NIST:** NIST SP 800-204C — policy-as-code for microservices; NIST SP 800-137 — ISCM; NIST OSCAL — certification format.
- **Tech:** OPA (openpolicyagent.org), Neo4j (neo4j.com), Backstage (backstage.io), Grafana (grafana.com), Sigstore (sigstore.dev), OSCAL (pages.nist.gov/OSCAL), OpenTelemetry (opentelemetry.io), SLSA (slsa.dev), Flux (fluxcd.io).

Chapter 2 — Architectural Overview

2.1 The C4 Context Diagram

The RC2 architecture is described in the C4 model (Simon Brown): Context, Container, Component, and Code. This section presents the Context level, which situates the system in its environment, identifying the external actors (people and systems) with which the RC2 platform interacts. The Context diagram is the highest level of the architecture; it does not show internal components, only the system boundary and the actors that cross it.

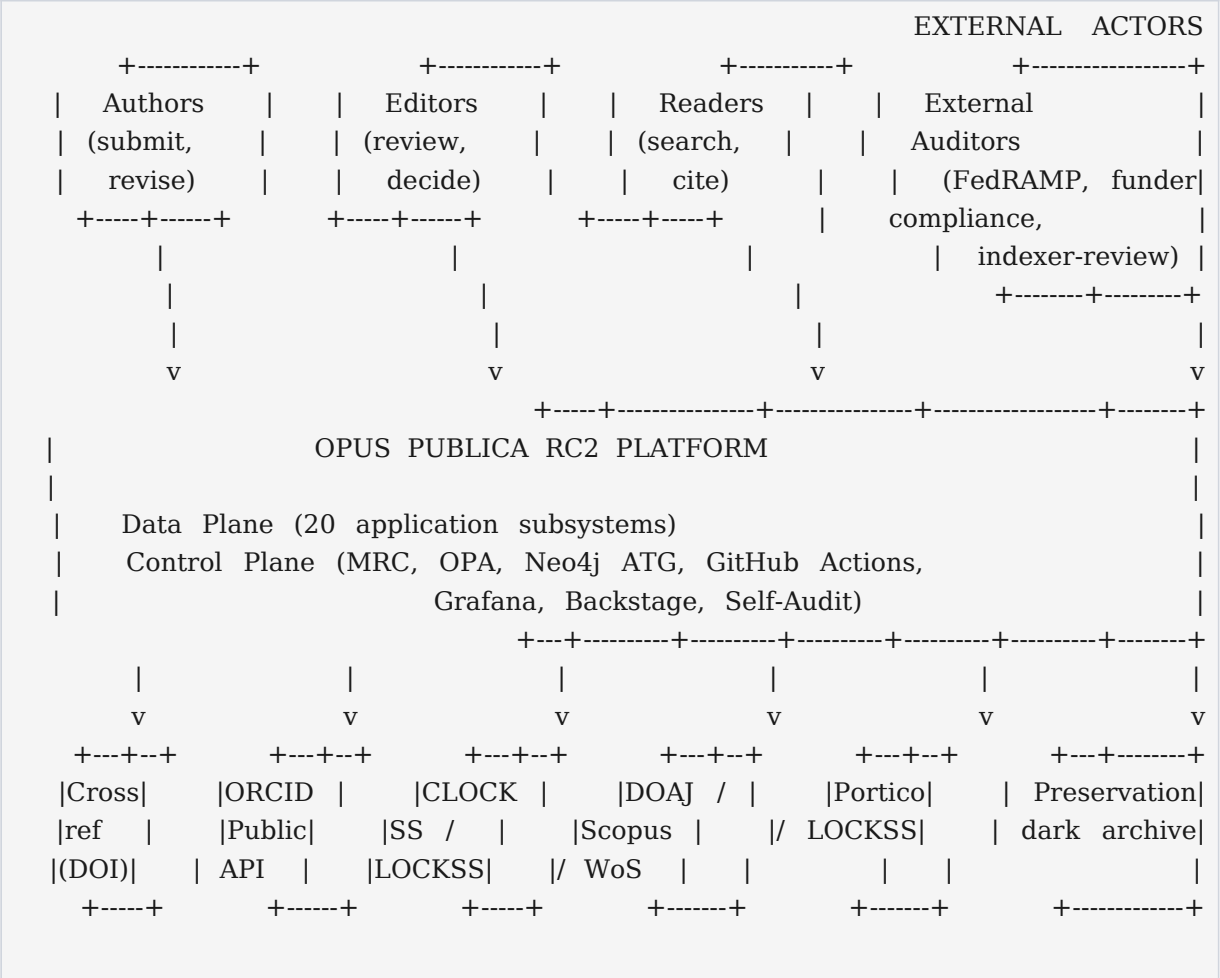
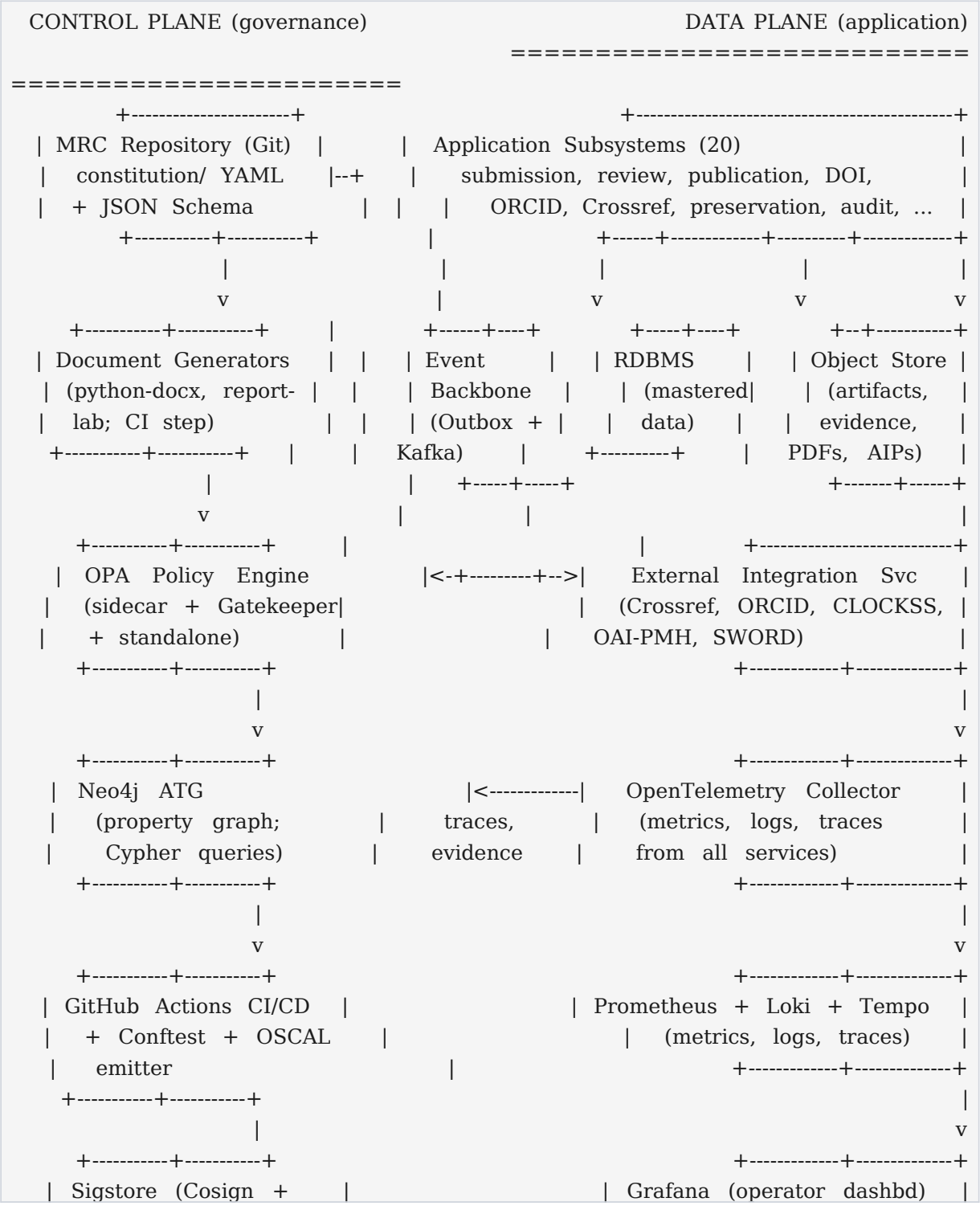


Figure 2.1: C4 Context Diagram — RC2 Platform and External Actors

The Context diagram identifies four classes of human actor (authors, editors, readers, external auditors) and six classes of external system (Crossref for DOI registration, ORCID for author identity, CLOCKSS/LOCKSS for preservation, DOAJ/Scopus/WoS for indexer eligibility, Portico for additional preservation, and the preservation dark archive). Every external interaction is governed by a contract in the MRC (Volume II Chapter 27) and is enforced by an OPA policy at the boundary service. The RC2 platform's system boundary is the dashed rectangle; everything inside it is the subject of this specification.

2.2 The Container Diagram

The Container diagram (C4 Level 2) shows the major deployable units of the RC2 platform — the containers, services, data stores, and infrastructure components that together implement the system. The diagram distinguishes the control plane (governance components, drawn with double-line borders) from the data plane (application components, drawn with single-line borders). The separation is a foundational architectural principle (Section 2.3) and is enforced by Kubernetes network policy (Chapter 7).



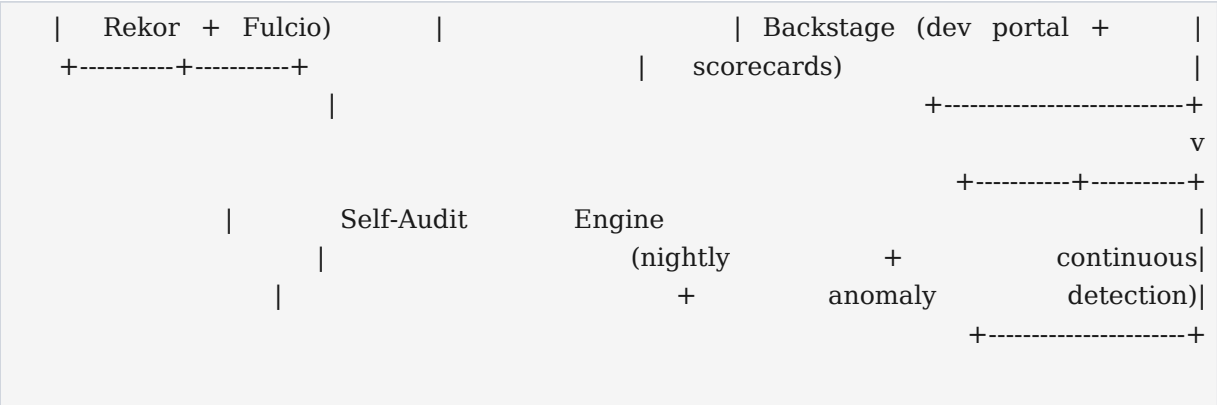


Figure 2.2: C4 Container Diagram — RC2 Control Plane and Data Plane

The container diagram conveys the architectural shape: the control plane is a pipeline from the MRC repository through policy evaluation, traceability graph construction, CI execution, signing, and audit, with the results surfaced on Grafana and Backstage. The data plane is a set of application services that emit events to the backbone, persist to RDBMS and object storage, and integrate with external systems; all data-plane activity is observed by the OpenTelemetry collector, which feeds the observability stack and the ATG. The control plane governs the data plane by three mechanisms: (1) Conftest gates that block a PR from merging if it violates a provision; (2) OPA Gatekeeper admission webhooks that block a deployment that violates a provision; (3) OPA sidecars that enforce policy at runtime within each service.

2.3 Architectural Principles

The RC2 architecture is governed by five architectural principles. These principles are normative: where a design choice is ambiguous, the choice that better serves these principles is the choice that SHALL be made. The principles are listed in priority order; when principles conflict, the higher principle prevails.

#	Principle	Operationalization
1	Policy-as-Code	Every constitutional provision that can be expressed as a policy SHALL be expressed as a Rego policy in the MRC, evaluated by OPA at PR time (Conftest), deploy time (Gatekeeper), and runtime (sidecar). NIST SP 800-204C defines the discipline; OPA/Rego is the implementation.
2	GitOps	Git SHALL be the source of truth for system state: the MRC for provisions, the application repo for code, the infrastructure repo for Kubernetes manifests, and the certification repo for evidence. Flux

		reconciles live state to declared state; drift is a defect. CNCF OpenGitOps principles.
3	Shift-Left Verification	Verification SHALL occur at the earliest possible point in the lifecycle: schema validation at commit time, policy evaluation at PR time, admission control at deploy time, runtime monitoring in production. Defects caught at PR time cost minutes; defects caught at release time cost hours; defects caught in production cost days.
4	Continuous Assurance	Compliance SHALL be a continuous property of the running system, verified every commit, every deploy, and every night, rather than a periodic state verified at release milestones. ISACA's continuous-assurance framing; NIST SP 800-137 ISCM 6-phase lifecycle.
5	Control-Plane / Data-Plane Separation	The governance components (control plane) SHALL be physically and logically separated from the application components (data plane). The control plane enforces policy on the data plane; the data plane does not enforce policy on the control plane. Network policy (Chapter 7) enforces the separation at the Kubernetes level.

2.4 Non-Functional Requirements

The RC2 architecture is bound by the following non-functional requirements (NFRs). The NFRs are derived from the RC1 Constitution Volume II Chapter 32 (SLOs) and Chapter 31 (thresholds) and are operationalized by the observability architecture (Chapter 9). NFRs are stated as measurable targets; an architecture that does not meet an NFR is defective.

Category	NFR	Target	Verification
Availability	Control-plane uptime	99.95% monthly (SLO-01).	Prometheus SLO burn;

			error budget alerting.
Availability	Data-plane uptime (publication path)	99.9% monthly (SLO-02).	Prometheus SLO burn; runbook on breach.
Performance	OPA decision latency (p99)	≤ 20 ms (THRESH-LAT-01).	OPA decision log histogram; alert on exceed.
Performance	PR pre-merge gate total time	≤ 10 min for 95% of PRs.	GitHub Actions timing; weekly report.
Performance	ATG query latency (p99, 2-hop)	≤ 500 ms (THRESH-LAT-04).	Neo4j query log; alert on exceed.
Performance	Crossref deposit latency (median)	≤ 5 min from publication event.	Crossref API poll; reconciliation job.
Scalability	Peak sustained submission rate	100 submissions/hour with linear scale.	Load test (k6) before each release.
Scalability	ATG size (nodes / edges)	≤ 5 M nodes / 50 M edges without degradation.	Neo4j store size monitor.
Scalability	Audit log retention	7 years (legal hold), tamper-evident.	Loki + S3 Object Lock; nightly hash chain.
Security	Supply-chain integrity level	SLSA L3 for every production artifact.	slsa-github-generator; Rekor log.
Security	Time-to-detect invariant violation	≤ 1 hour (continuous checks).	Self-audit; SEV-1 page on detection.
Security	Time-to-detect audit-log anomaly	≤ 5 min (rule-based); ≤ 24 h (ML).	Anomaly detection; ticket / page.
Observability	Trace coverage	100% of publish-path requests traced.	OTel SDK; sampled at 100% for critical path.
Observability	Metric freshness	≤ 15 s scrape interval; ≤ 60 s dashboard lag.	Prometheus scrape config; Grafana.
Recoverability	Recovery Time Objective (RTO)	≤ 4 h for full platform restoration.	DR drill quarterly; runbook-tested.
Recoverability	Recovery Point Objective (RPO)	≤ 15 min (continuous WAL replication).	PostgreSQL + S3 cross-region replication.

Chapter 3 — Control Plane: The Governance Subsystem

The control plane is the governance subsystem of RC2. It comprises the components that enforce the Constitution: the Machine-Readable Constitution repository, the OPA policy engine, the Neo4j traceability graph, the GitHub Actions continuous certification engine, the Grafana and Backstage governance dashboard, and the self-audit engine. This chapter specifies each component at the architectural level; the per-component detailed specifications (Purpose, Responsibilities, Inputs, Outputs, Dependencies, Interfaces, Deployment, Failure Modes, Runtime Verification) are in Chapter 5.

3.1 The Machine-Readable Constitution (MRC) Repository

The MRC repository is the single source of truth for every constitutional provision. It is a Git repository (opus-publica/constitution on GitHub) whose directory tree mirrors the provision class hierarchy: invariants/, rules/, contracts/, fields/, events/, slos/, thresholds/, state-machine/, certification/, schemas/, policies/, and generators/. Each provision is a YAML file conforming to a JSON Schema 2020-12 definition in schemas/. Each Rego policy that enforces a provision lives in policies/. The generators/ directory contains Python scripts that read the MRC and produce every human-readable document (Constitution Vol I and Vol II, Playbook, Traceability Matrix, Certification Checklist) plus the OSCAL Profile. The repository is the canonical source; the documents are derived views.

The MRC's validation pipeline runs on every commit and every PR. The pipeline consists of: (1) YAML linting (yamllint); (2) JSON Schema 2020-12 validation (jsonschema CLI) of every provision against its schema; (3) cross-reference validation (a custom Rego policy that confirms every referenced provision, ADR, code module, and test exists in its declared location); (4) Rego unit tests (opa test) for every policy module; (5) document generation (every generator runs and the regenerated documents are committed alongside the provision change); (6) constitutional diff (a structured JSON Patch RFC 6902 produced from the before/after MRC, with a compatibility classification BACKWARD/FORWARD/FULL/NONE per the Confluent Schema Registry model). A PR that fails any step is blocked; the failure message includes the offending file, the schema line, and the regeneration command the engineer can run locally to reproduce.

3.2 The Policy Engine (OPA)

The Open Policy Agent (OPA) is the policy engine of RC2. OPA is a CNCF Graduated project that evaluates policies written in Rego, a declarative policy language, against JSON data. In RC2, OPA is deployed in three configurations, each serving a distinct enforcement point:

- **PR-time (Conftest):** runs as a CI step (via Conftest, an OPA subproject) on every PR, evaluating every Rego policy whose provisions the PR touches. The policy input is the PR's diff (parsed as YAML/JSON/HCL) plus the MRC's relevant provisions. A FAIL blocks the PR; the FAIL message links to the provision and the evidence required.

- **Deploy-time (Gatekeeper):** deployed as OPA Gatekeeper, a Kubernetes admission webhook that evaluates Rego policies on every admission request (create, update, delete) to the platform's Kubernetes namespaces. A FAIL rejects the request; the FAIL message links to the provision. Gatekeeper enforces the platform's own infrastructure (the control plane governs itself).
- **Runtime (sidecar):** deployed as a sidecar container in every application service pod. The service queries the sidecar via OPA's REST API before performing a policy-governed action (e.g., before minting a DOI, the publication service queries OPA with the article's metadata; OPA evaluates INV-I-02 and returns ALLOW or DENY). The sidecar's bundle is updated every 30 s from the bundle server, which compiles the MRC's policies/ directory.

OPA's bundle management is centralized: a bundle server (a simple HTTP service) compiles the MRC's policies/ directory into a bundle.tar.gz signed with Cosign, and serves it to every OPA instance (Conftest pulls at CI time; Gatekeeper pulls via its configured bundle source; sidecars pull every 30 s). Bundle signature verification (Cosign) is mandatory: an OPA instance that cannot verify the bundle signature SHALL refuse to load the bundle and SHALL fail closed (deny all decisions). Decision logging is centralized: every OPA decision (input, policy ID, result, timestamp, decision ID) is shipped to the audit log (Loki) and to the ATG as an Evidence node, providing end-to-end traceability from provision to policy to decision to action.

3.3 The Traceability Graph (Neo4j)

The Automated Traceability Graph (ATG) is implemented on Neo4j, a property-graph database. Neo4j is chosen because traceability queries are graph traversals (find every test that verifies a requirement realized by a component that depends on service X), and graph traversals are O(depth) in a graph database but O(joins^depth) in a relational database. The reference implementation is Neo4j 5.x Community Edition (open source) or Enterprise Edition (for clustering and Role-Based Access Control); Amazon Neptune and Memgraph are evaluated in ADR-008 as managed-service alternatives.

The ATG's schema consists of 13 node types (Requirement, Architecture, ADR, Database, API, Code, Storage, Workflow, UIComponent, Test, Evidence, Certification, Release) and 12 edge types (realizes, justified-by, backed-by, exposed-by, implemented-by, stored-in, exercised-by, surfaced-by, verified-by, produces, certified-by, released-in). Every node carries the provision's id and metadata (severity, owner, version) from the MRC; every edge carries a type, a source (machine-detected, declared, inferred), a timestamp, and a confidence (high, medium, low). The schema is enforced by Neo4j constraints (uniqueness on node id, existence on required properties) and by a Rego policy that validates every graph mutation proposed by a builder.

The ATG is built by three builders (Section 5.9) that run in CI on every commit and every release: the Constitution Builder (Requirement nodes from the MRC), the Code Builder (Architecture, ADR, Database, API, Code, Storage nodes from AST analysis and annotation parsing), and the CI Builder (Test, Evidence, Certification, Release nodes from CI metadata). Edges are produced by the same builders from code annotations (@requires, @traces-to), OpenAPI x-traces-to extensions, and CI metadata. The integrity of the ATG is verified by eight integrity rules (IR-01 through IR-08) implemented as Cypher queries that run

after every build: orphan requirements (IR-01), orphan APIs (IR-02), orphan tests (IR-03), missing evidence (IR-04), broken edges (IR-05), stale evidence (IR-06), degree-zero nodes (IR-07), and low-confidence edges (IR-08). A violation fails the build; the build cannot merge until the violation is resolved or a documented exception is recorded.

3.4 The Continuous Certification Engine (GitHub Actions + Conftest + OSCAL)

The Continuous Certification Engine (CCE) is the integrated set of GitHub Actions workflows, Conftest evaluations, and OSCAL emitters that evaluate every certification criterion continuously. The CCE has three cadences:

- **Per-PR:** every PR triggers a workflow that runs every Rego policy whose provisions the PR touches (determined by an ATG query). The PR's certification preview is a per-PR OSCAL Assessment Results fragment that summarizes which criteria PASS, FAIL, or are NOT_APPLICABLE.
- **Nightly:** every night at 02:00 UTC, a workflow runs every criterion that has a runtime component (e.g., invariant checks that query the production database). The nightly OSCAL-AR is committed to the certification repository and surfaced on Grafana.
- **Per-release:** every release triggers the Production Acceptance Gate workflow, which evaluates every GATE criterion (criterion marked gate: true in the MRC) and emits the release's OSCAL-AR. The RCA's sign-off is a GitHub Environment approval that depends on the Gate workflow passing.

The OSCAL Assessment Results emitter is a Python script (generators/gen_oscal_ar.py) that reads the CI run results (from the GitHub Actions API) and the MRC's certification/criteria/ directory, and produces an OSCAL Assessment Results JSON document conforming to the NIST OSCAL Assessment Results model. The OSCAL-AR is the machine-readable certification record: it identifies the system (Opus Publica RC2), the assessment (PR/nightly/release), the controls (criteria), the results (PASS/FAIL/NOT_APPLICABLE), the evidence (links to the ATG Evidence nodes), and the assessor (the CI job). The OSCAL-AR is committed to the certification repository, indexed by the ATG, surfaced on Grafana, and made available to external auditors via a signed feed.

3.5 The Governance Dashboard (Grafana + Backstage)

The governance dashboard is bifurcated by audience. Grafana (open-source visualization platform) is the operator-facing surface; it is the dashboard the on-call engineer consults during an incident and the RCA consults during a release review. Backstage (Spotify open-source developer portal; CNCF) is the developer-facing surface; it is the portal the engineer consults to see their service's compliance posture and the scaffolder they use to create a new compliant service. The two surfaces consume the same data sources (Prometheus, Loki, Tempo, Neo4j, the OSCAL-AR feed) but present them at different levels of abstraction: Grafana is operational (alerts, SLOs, metrics), Backstage is developmental (catalog, scorecards, scaffolder, docs).

The Grafana Constitutional Health Dashboard has panels for: (1) Invariant Health (38 invariants, current status, last-verified timestamp); (2) SLO Burn (10 SLOs, error budget consumed, projected breach date);

(3) Certification Coverage (criterion count by status, drift from last release); (4) Audit Log Anomalies (rule-based alerts, ML anomalies); (5) Supply Chain Integrity (SLSA L3 verification status, Rekor log entries); (6) ATG Integrity (the 8 integrity rules, pass/fail per rule). The Backstage Scorecards have one card per service, with pillars (e.g., “Has Catalog Info”, “Has SLOs”, “Has Traces”, “Has Recent Evidence”, “Passes All GATE Criteria”) and a grade (A through F). The scorecard data comes from the ATG (which has every artifact linked) and from the OSCAL-AR feed (which has every criterion's status).

3.6 The Self-Audit Engine

The Self-Audit Engine (SAE) is the operational form of continuous assurance (ISACA) and Information Security Continuous Monitoring (NIST SP 800-137). The SAE runs at two cadences: nightly (a GitHub Actions workflow at 02:00 UTC that runs every self-audit check and produces a nightly audit report) and continuous (real-time rule-based anomaly detection on the audit log, with ML-based anomaly detection on a 15-minute window). The SAE's checks are:

- **Traceability integrity:** ATG queries (orphan requirements, orphan APIs, broken edges, stale evidence, drift).
- **Certification gaps:** OSCAL-AR queries (criteria not PASS in last 24h; criteria with stale evidence).
- **Drift detection:** driftctl scan (live cloud state vs. Terraform state); Rego policy (live contract vs. MRC-pinned contract).
- **Vulnerability and secret scan:** CycloneDX + vulnerability DB; secret manager query (expired credentials).
- **Anomaly detection:** rule-based (Rego policies evaluated on every audit record) + ML-based (Isolation Forest and Local Outlier Factor on the audit log, 15-minute window).

The SAE's findings are classified SEV-1 through SEV-3. SEV-1 findings page the on-call engineer immediately and block the next release; SEV-2 findings open a high-priority ticket and block the next release; SEV-3 findings open a normal ticket and do not block. The nightly report is committed to the certification repository, summarized on Grafana, and is the input to the weekly CSA review. The continuous findings (anomaly detection) page the on-call engineer (rule-based SEV-1/2) or open a ticket (ML-based, lower confidence) for human review. The SAE's two layers — rule-based and ML-based — are complementary: rules catch the known; ML catches the novel; together they provide defense-in-depth for the audit log.

Chapter 4 — Data Plane: The Application Subsystem

The data plane is the application subsystem of RC2: the components that produce and consume user-visible data. The data plane is governed by the control plane (every application service is deployed behind Gatekeeper, every service has an OPA sidecar, every service emits OpenTelemetry traces) but the data plane's internal architecture is the subject of this chapter. The data plane comprises the 20 application subsystems, the event backbone, storage, and external integrations.

4.1 Application Services

The 20 application subsystems are specified in Constitution Volume II Chapter 27. They are the bounded contexts of the Opus Publica platform, each owning a discrete capability and a discrete data store. The subsystems are deployed as independent Kubernetes Services, each behind an Ingress, each with its own RDBMS schema (or a logical partition of the shared RDBMS), and each emitting domain events to the event backbone. The 20 subsystems are summarized below.

#	Subsystem	Owns	Consumes From
1	Submission	Author submissions, files, metadata.	Identity (auth).
2	Review	Reviews, decisions, reviewer assignments.	Submission, Identity.
3	Publication	State machine, versioning, AIP.	Review, DOI, ORCID.
4	DOI Minter	DOI registration, Crossref deposit.	Publication.
5	ORCID Sync	Author ORCID linking, auto-update.	Identity, Publication.
6	Preservation	CLOCKSS/LOCKSS/Portico deposit.	Publication.
7	Search	Full-text search, discovery.	Publication.
8	Citation	Citation graph, citation counts.	Publication, Crossref.
9	Audit	Immutable audit log, hash chain.	All subsystems (events).
10	Notification	Email, in-app notifications.	All subsystems.
11	Identity	Users, roles, OIDC	(none; root).

		sessions.	
12	Authorization	RBAC, OPA enforcement points.	Identity, OPA.
13	Reporting	Usage stats, editorial reports.	All subsystems.
14	Editorial	Editorial workflow, assignments.	Submission, Review.
15	Licensing	License selection, CC enforcement.	Submission, Publication.
16	Embargo	Embargo periods, lift timing.	Publication.
17	Correction	Errata, retractions, expressions.	Publication, Audit.
18	Export	OAI-PMH, SWORD, JATS XML.	Publication.
19	Import	Bulk import, migration tools.	Submission, Identity.
20	Storage	Object storage abstraction.	All subsystems (files).

4.2 The Event Backbone

The event backbone is the mechanism by which subsystems communicate asynchronously. RC2 uses the transactional outbox pattern: a subsystem writes to its RDBMS and to an outbox table in the same transaction; a separate process (Debezium, or a custom poller) reads the outbox and publishes to Kafka (or Redpanda, a Kafka-compatible alternative evaluated in ADR-009). The outbox pattern guarantees that an event is published if and only if the transaction commits, eliminating the dual-write problem (event published but transaction rolled back, or vice versa).

Kafka (or Redpanda) is the event broker. Each event type (34 events specified in Constitution Volume II Chapter 29, e.g., ArticleSubmitted, ArticlePublished, DOIMinted, PreservationDeposited) has a dedicated Kafka topic with a defined schema (JSON Schema 2020-12, registered in the Confluent Schema Registry for compatibility management). Consumers are organized into consumer groups; the event is delivered to one consumer per group, with at-least-once semantics and consumer-side idempotency (every event has an idempotency key, and consumers deduplicate on the key). The Audit subsystem (subsystem 9) is a consumer of every topic; it appends every event to the immutable audit log, where the event becomes evidence for the ATG.

4.3 Storage

RC2 uses three storage tiers, each suited to a different access pattern:

- **RDBMS (PostgreSQL):** for mastered data (articles, authors, journals, submissions, reviews). PostgreSQL 16 is the reference implementation; the database is operated as a primary with synchronous streaming replication to a cross-region standby (RPO ≤ 15 min). Each subsystem owns its schema; cross-subsystem access is via API, not via shared tables.
- **Object storage (S3 / MinIO):** for large binary artifacts (submission files, published PDFs, Archival Information Packages, evidence files). S3 (or MinIO for self-hosted) is the reference implementation; objects are versioned, encrypted at rest (KMS), and replicated cross-region. Published artifacts are additionally protected by S3 Object Lock (Compliance mode, 7-year retention) to satisfy the immutability invariant (INV-M-01).
- **Graph DB (Neo4j):** for the Automated Traceability Graph (Chapter 3.3). Neo4j 5.x is the reference implementation. The graph store is operated as a causal cluster (one leader, two followers) with daily full backups and continuous incremental backups.

4.4 External Integrations

RC2 integrates with six classes of external system, each via a dedicated integration service that owns the protocol, the credentials, the retry policy, and the reconciliation. The integrations are:

Integration	Protocol	Direction	Owner
Crossref	REST (deposit + query)	Outbound (deposit), inbound (reconcile).	DOI Minter subsystem.
ORCID	REST (Member API for write; Public API for read).	Both directions.	ORCID Sync subsystem.
CLOCKSS	SFTP deposit of AIPs.	Outbound only.	Preservation subsystem.
LOCKSS	LOCKSS protocol (LCAP).	Outbound only.	Preservation subsystem.
Portico	SFTP deposit of AIPs.	Outbound only.	Preservation subsystem.
OAI-PMH	HTTP (OAI-PMH 2.0).	Inbound (we are the data provider).	Export subsystem.
SWORD	HTTP (SWORD 1.3 / 2.0).	Both directions (deposit + receive).	Export / Import subsystems.
Indexers (DOAJ, Scopus, WoS)	Varies (file deposit, API).	Outbound only.	Export subsystem.

Chapter 5 — Component Specifications

This chapter specifies the 16 components of the RC2 architecture at the level of detail required for implementation. Each component specification follows a uniform structure: Purpose, Responsibilities, Inputs, Outputs, Dependencies, Interfaces, Deployment, Failure Modes, Runtime Verification. The structure is normative: an implementation that omits or deviates from any field of its component specification is defective and SHALL be corrected before release.

5.1 MRC Repository

The Machine-Readable Constitution repository is the single source of truth for every constitutional provision. It is a Git repository on GitHub (opus-publica/constitution) whose contents are YAML provision files, JSON Schema definitions, Rego policy modules, and Python document generators. The repository is the foundation of the control plane; every other control-plane component reads from it or from artifacts derived from it.

Aspect	Specification
Purpose	Be the single, versioned, machine-readable source of truth for every constitutional provision.
Responsibilities	Store provisions as YAML conforming to JSON Schema 2020-12; store Rego policies that enforce provisions; store generators that produce human-readable documents; validate every commit; produce a constitutional diff for every PR.
Inputs	Provision YAML files (authored by engineers); JSON Schema files; Rego policy files; Python generator scripts; PRs that modify any of the above.
Outputs	Validated MRC at every commit; regenerated .docx/.pdf documents committed alongside provision changes; JSON Patch (RFC 6902) constitutional diff for every PR; signed OPA bundle (policies/ + schemas/) for the bundle server; OSCAL Profile (generated).
Dependencies	GitHub (hosting); yamllint, jsonschema CLI (validation); opa (policy testing); python-docx, reportlab (document generation); Cosign (bundle signing); Confluent Schema Registry compatibility model (diff classification).
Interfaces	Git over HTTPS/SSH (clone, fetch, push); GitHub REST API (PR status, check runs); bundle server HTTPS

	endpoint (signed bundle); downstream consumers (ATG Constitution Builder, OPA bundle server, document generators).
Deployment	GitHub repository (managed). Branch protection: main is protected; PRs require 1 review and all CI checks passing. Tagged releases (SemVer) for every MRC version; releases are immutable.
Failure Modes	(1) Schema validation fails → commit rejected, engineer fixes YAML. (2) Rego policy test fails → commit rejected, engineer fixes policy. (3) Document generator fails → commit rejected, engineer fixes generator or input. (4) Bundle signing fails → bundle server serves stale bundle; OPA instances refuse to load new bundle and fail closed. (5) Repository unavailable (GitHub outage) → CI gates block all PRs; break-glass procedure (CSA authorization) documented.
Runtime Verification	CI status check on every PR (must pass); nightly self-audit (repository reachable, latest tag valid, bundle signature verifiable); Grafana panel (last commit timestamp, last release timestamp, open PR count by status).

5.2 Document Generators

The document generators are Python scripts (gen_constitution_vol1.py, gen_constitution_vol2.py, gen_playbook.py, gen_traceability_matrix.py, gen_certification_checklist.py, gen_oscal.py) that read the MRC and produce the human-readable .docx/.pdf documents and the OSCAL Profile. The generators run as a CI step on every MRC commit; the regenerated documents are committed alongside the provision change, guaranteeing that human-readable documents are never out of sync with the machine-readable source.

Aspect	Specification
Purpose	Produce every human-readable document and machine-readable certification profile from the MRC, eliminating hand-maintained cross-references and document drift.
Responsibilities	Read the MRC; apply the document-specific template; produce .docx (via python-docx) and .pdf (via reportlab); produce OSCAL Profile JSON; commit the

	regenerated documents to the MRC repo on every provision change.
Inputs	The MRC (constitution/ directory); document templates (Jinja2); traceability edges (for the Traceability Matrix generator, from the ATG export); CI run results (for the OSCAL Assessment Results generator, from the GitHub Actions API).
Outputs	Constitution Vol I/II (.docx, .pdf); Engineering Playbook (.docx, .pdf); Traceability Matrix (.docx, .pdf); Certification Checklist (.docx, .pdf); OSCAL Profile JSON; OSCAL Assessment Results JSON (per PR/nightly/release).
Dependencies	Python 3.11+; python-docx; reportlab; Jinja2; oscal-cli (NIST OSCAL tooling for validation); the MRC repo.
Interfaces	CI job (triggered on MRC commit); reads MRC from filesystem; writes artifacts to the MRC repo (committed) and to the certification repository (for OSCAL-AR).
Deployment	Runs in GitHub Actions (Ubuntu runner, Python 3.11). No persistent process; triggered by commit. Generator versions are pinned in the workflow file; updates to generators are themselves PRs.
Failure Modes	(1) Generator crashes on malformed MRC → CI fails, engineer fixes MRC or generator. (2) Generator produces invalid OSCAL → oscal-cli validation fails, CI fails. (3) Generator produces document that differs from hand-curated original (initial migration only) → engineer reviews diff, accepts generator output, retires hand-curated original.
Runtime Verification	CI check (generator must succeed and output must validate); nightly self-audit (every document's hash matches the MRC's expected hash); Grafana panel (last generation timestamp, generator duration, generator failure rate).

5.3 OPA Policy Engine

Open Policy Agent (OPA) is the policy engine of RC2. OPA evaluates policies written in Rego against JSON data, returning ALLOW or DENY decisions. In RC2, OPA is deployed in three configurations: PR-time

(Conftest), deploy-time (Gatekeeper), and runtime (sidecar). The three configurations share a common bundle, signed and served by the bundle server.

Aspect	Specification
Purpose	Evaluate constitutional provisions as executable Rego policies at PR, deploy, and runtime; be the single enforcement point for policy-as-code in RC2.
Responsibilities	Load the signed bundle; evaluate policies on request; return ALLOW/DENY with the matched policy ID and provision reference; log every decision to the audit log; ship every decision to the ATG as an Evidence node.
Inputs	Signed bundle from the bundle server (Rego policies + MRC data); policy query (input JSON from the caller).
Outputs	Decision (allow: boolean; package: string; rule: string; decision_id: UUID; timestamp: ISO 8601); decision log entry; ATG Evidence node.
Dependencies	OPA 0.60+ (CNCF Graduated); the bundle server; the audit log (Loki); the ATG (Neo4j, via the CI Builder); Cosign (bundle signature verification).
Interfaces	REST API (POST /v1/data/<package> with input JSON, returns decision JSON); bundle fetch (HTTPS GET from bundle server every 30 s for sidecars, on-demand for Conftest); decision log ship (HTTPS POST to Loki).
Deployment	(1) Conftest: GitHub Actions step, ephemeral. (2) Gatekeeper: Kubernetes deployment in the gatekeeper-system namespace, 3 replicas, admission webhook configured. (3) Sidecar: sidecar container in every application pod, 1 per pod, bundle fetched every 30 s.
Failure Modes	(1) Bundle signature verification fails → OPA refuses to load bundle, fails closed (deny all). (2) Bundle fetch fails (network) → OPA serves stale bundle (up to 24 h); after 24 h, fails closed. (3) Decision log ship fails → OPA retries with exponential backoff; after 100 retries, decision log buffered locally and shipped on recovery. (4) Rego policy crashes → OPA returns a decision-error, treated as DENY.
Runtime Verification	Prometheus metric opa_decision_total (counter by policy ID, result); opa_bundle_load_timestamp (gauge, alert if > 35 s stale);

	opa_decision_log_ship_failures_total (counter, alert if > 0); nightly self-audit (every Rego policy has at least one unit test, every policy has a provision reference, every provision with a policy has the policy).

5.4 Conftest Pre-Merge Runner

Conftest is an OPA subproject for testing structured configuration data (YAML, JSON, HCL, INI, Dockerfile, etc.) against Rego policies. In RC2, Conftest runs as a GitHub Actions step on every PR, evaluating every Rego policy whose provisions the PR touches. The PR's certification preview is a per-PR OSCAL Assessment Results fragment summarizing the criteria PASS/FAIL/NOT_APPLICABLE.

Aspect	Specification
Purpose	Enforce every constitutional provision that can be evaluated from PR content at PR time, before the PR merges, blocking merge on failure.
Responsibilities	Determine which provisions the PR touches (ATG query); fetch the relevant Rego policies and MRC data from the bundle; run Conftest on the PR's diff; emit a per-PR OSCAL-AR fragment; report results to the GitHub Check UI.
Inputs	PR diff (from GitHub Actions); PR metadata (files, author, branch); the OPA bundle (policies + MRC data); the ATG query result (which provisions the PR touches).
Outputs	GitHub Check run (PASS/FAIL with per-policy detail); per-PR OSCAL-AR fragment (committed to the certification repository); ATG Evidence nodes (one per policy evaluation).
Dependencies	Conftest (OPA subproject); the OPA bundle; the ATG (Neo4j, for the touch-query); the GitHub Actions API; the certification repository.
Interfaces	GitHub Actions workflow (triggered on PR); reads PR diff from GitHub Actions workspace; reads bundle from bundle server; queries ATG via Cypher over HTTPS; writes OSCAL-AR fragment to certification repo.
Deployment	GitHub Actions workflow (Ubuntu runner, pre-merge job). Required check on main: a PR cannot merge until the Conftest check passes.

Failure Modes	(1) ATG touch-query fails → runner evaluates all policies (over-approximation, slower but safe). (2) Bundle fetch fails → runner uses cached bundle (max 24 h); after 24 h, runner fails the PR with an actionable error. (3) Conftest crashes → PR fails with crash trace; runner is fixed. (4) False-positive block (policy too strict) → engineer files an MRC PR to relax the policy or marks the criterion N/A with RCA justification.
Runtime Verification	GitHub Actions timing (workflow duration, alert if p99 > 10 min); per-policy PASS/FAIL rate (alert if FAIL rate > baseline); nightly self-audit (every active policy has run on at least one PR in the last 7 days, every PR's OSCAL-AR fragment is in the certification repo).

5.5 GitHub Actions CI/CD

GitHub Actions is the CI/CD substrate of RC2. Every workflow — pre-merge Conftest, document generation, ATG build, nightly self-audit, release certification, supply-chain signing — is a GitHub Actions workflow. GitHub Actions is chosen because it is the platform the engineering organization already uses, because it integrates natively with the GitHub-hosted MRC and application repositories, and because its Environment approval gates provide the RCA's release sign-off mechanism.

Aspect	Specification
Purpose	Execute every CI/CD workflow in RC2: pre-merge gates, document generation, ATG build, nightly self-audit, release certification, supply-chain signing.
Responsibilities	Trigger workflows on the correct events (PR, push, cron, release); execute workflow steps in ephemeral runners; manage secrets via GitHub Secrets and OIDC tokens for cloud access; enforce Environment approval gates for production deployments.
Inputs	Workflow YAML files (in .github/workflows/); event payloads (PR, push, schedule, release); secrets (from GitHub Secrets); OIDC tokens (from GitHub OIDC provider).
Outputs	Workflow run results (success/failure, logs, artifacts); artifacts (test reports, evidence files, OSCAL-AR fragments, signed images); GitHub Check runs (per-job status).
Dependencies	GitHub (hosting, runners, OIDC); self-hosted runners

	for jobs that need VPC access (ATG build, driftctl scan); the OPA bundle; the ATG; the certification repository; the container registry.
Interfaces	GitHub Actions API (workflow trigger, run status, artifact download); GitHub Secrets (per-repo, per-environment); GitHub Environments (staging, production, with required reviewers); GitHub OIDC (for cloud role assumption).
Deployment	GitHub-hosted runners (default); self-hosted runners (a Kubernetes-based runner set in the platform VPC for VPC-access jobs). Runner autoscaling via actions-runner-controller.
Failure Modes	(1) GitHub Actions outage → no workflows run; break-glass procedure (CSA authorization, documented) permits the RCA to authorize a release with recorded justification. (2) Self-hosted runner pool exhausted → jobs queue (alert at queue depth > 10). (3) Secret rotation fails → jobs using the secret fail; alert on secret-expiry T-minus-7-days. (4) OIDC trust broken → cloud role assumption fails; alert immediately.
Runtime Verification	GitHub Actions status API (workflow success rate, alert if < 95% over 24 h); workflow duration (alert if p99 > 2x baseline); runner pool utilization (alert if > 80%); nightly self-audit (every required workflow has run in the last 24 h, every workflow's secrets are valid for > 7 days).

5.6 OPA Gatekeeper Admission Controller

OPA Gatekeeper is a Kubernetes admission webhook that enforces Rego policies on every admission request (create, update, delete) to the platform's Kubernetes namespaces. Gatekeeper is the deploy-time enforcement point: where Conftest enforces at PR time, Gatekeeper enforces at deploy time, ensuring that a deployment that bypasses CI (e.g., a manual kubectl apply) is still governed.

Aspect	Specification
Purpose	Enforce constitutional provisions on Kubernetes admission requests, providing deploy-time policy enforcement independent of CI.
Responsibilities	Intercept admission requests; evaluate the relevant Constraint policies; ALLOW or DENY the request; log

	every decision; report violations to the audit log and ATG.
Inputs	Admission request (Kubernetes object: Deployment, Service, ConfigMap, Secret, etc.); Constraint policies (Rego, from the Gatekeeper constraint framework); ConstraintTemplate instances (the policy parameters).
Outputs	Admission response (ALLOW/DENY with reason); audit log entry; ATG Evidence node.
Dependencies	Gatekeeper 3.14+; Kubernetes 1.27+; the OPA bundle (for Constraint policies); the audit log (Loki); the ATG (Neo4j).
Interfaces	Kubernetes admission webhook (ValidatingWebhookConfiguration); ConstraintTemplate and Constraint CRDs (the policy API); audit log ship (HTTPS POST to Loki).
Deployment	Kubernetes deployment in the gatekeeper-system namespace, 3 replicas, HA. Webhook configured with failurePolicy: Fail (fail closed if Gatekeeper is unavailable).
Failure Modes	(1) Gatekeeper unavailable → admission requests fail (failurePolicy: Fail); alert immediately. (2) Constraint policy crashes → admission request denied (fail safe). (3) Constraint drift (live constraints diverge from MRC-declared) → nightly driftctl scan detects; SEV-1 page. (4) False-positive block (constraint too strict) → engineer files an MRC PR or marks the constraint enforcementAction: warn (report-only) with RCA justification.
Runtime Verification	Prometheus metric gatekeeper_admission_total (counter by kind, allowed/denied); gatekeeper_admission_latency_seconds (histogram, alert if p99 > 100 ms); gatekeeper_violation_total (counter, alert if > baseline); nightly self-audit (every Gatekeeper constraint has a provision reference, every active provision with a deploy-time policy has the constraint).

5.7 Flux GitOps Controller

Flux is a CNCF Graduated GitOps controller that reconciles live Kubernetes state to declared state in Git. In RC2, every Kubernetes manifest (Deployment, Service, ConfigMap, Secret, etc.) lives in the infrastructure repository (opus-publica/infrastructure); Flux watches the repository and applies changes to the cluster. Flux is the deploy mechanism: there is no manual kubectl apply in RC2; every change is a PR to the infrastructure repo, reviewed, merged, and reconciled by Flux.

Aspect	Specification
Purpose	Reconcile live Kubernetes state to declared state in Git, providing the GitOps deploy model and eliminating manual cluster mutation.
Responsibilities	Watch the infrastructure repo; detect changes; apply changes to the cluster; report drift (live state diverges from declared state); emit events on reconciliation success/failure.
Inputs	Git repository (infrastructure repo, branches per environment); Kubernetes manifests (YAML, Kustomize, or Helm); image registry (for image updates).
Outputs	Reconciled cluster state; reconciliation events (success/failure, duration); drift reports; notifications (Slack, on failure).
Dependencies	Flux 2.x (CNCF Graduated); Kubernetes 1.27+; the infrastructure repository; the image registry; the OPA bundle (Gatekeeper enforces policies on Flux's applies).
Interfaces	Git over HTTPS/SSH (read infrastructure repo); Kubernetes API (apply manifests); notification API (Slack, on failure); image registry API (for image update automation).
Deployment	Flux controllers (source-controller, kustomize-controller, helm-controller, notification-controller, image-reflector-controller, image-automation-controller) in the flux-system namespace, 1 replica each (HA via leader election).
Failure Modes	(1) Flux cannot reach the Git repository → reconciliation stops; alert immediately. (2) Flux applies a manifest that Gatekeeper denies → reconciliation fails; alert; engineer fixes the manifest or the

	constraint. (3) Drift detected (live state diverges from declared) → drift report emitted; SEV-2 if drift is in a production namespace. (4) Flux controller crashes → leader election promotes a new leader; reconciliation resumes.
Runtime Verification	Prometheus metric flux_reconciliation_total (counter by kind, success/failure); flux_reconciliation_duration_seconds (histogram); flux_drift_detected_total (counter, alert if > 0 in production); nightly self-audit (every declared resource is present in the cluster, every cluster resource is declared in Git).

5.8 Neo4j Traceability Graph

The Automated Traceability Graph (ATG) is implemented on Neo4j, a property-graph database. The ATG is the live form of traceability: every engineering artifact (requirement, architecture component, ADR, database, API, code module, storage object, workflow, UI component, test, evidence, certification, release) is a node, and every traceability relationship is an edge. The ATG is queried by graph traversal to answer the auditor's question — prove it — with exhaustive, evidence-backed answers in milliseconds.

Aspect	Specification
Purpose	Be the live, machine-built property graph that replaces the RC1 hand-maintained Traceability Matrix and answers traceability queries by graph traversal.
Responsibilities	Store 13 node types and 12 edge types; enforce schema constraints (uniqueness, existence); accept graph mutations from the three builders; execute the 8 integrity rules after every build; serve graph queries (Cypher) to consumers (Conftest touch-query, Grafana, Backstage, Self-Audit).
Inputs	Graph mutations from the three builders (Constitution, Code, CI); Cypher queries from consumers.
Outputs	Graph query results (JSON); integrity rule reports (PASS/FAIL per rule); export snapshots (for the Traceability Matrix generator).
Dependencies	Neo4j 5.x (Community or Enterprise); the MRC (for the Constitution Builder); the codebase (for the Code Builder); CI metadata (for the CI Builder); the audit log (for evidence-node timestamps).

Interfaces	Cypher over Bolt (read/write queries); Neo4j HTTP API (for administrative operations); export API (CSV/JSON snapshot for the Traceability Matrix generator).
Deployment	Neo4j causal cluster (1 leader, 2 followers) in the control-plane namespace, persistent volumes (200 GB SSD), daily full backup + continuous incremental backup to object storage. Network policy restricts access to the control-plane namespace and to authorized data-plane query service accounts.
Failure Modes	(1) Leader fails → cluster elects a new leader (RTO < 30 s). (2) Cluster unavailable → builders buffer mutations and retry; consumers fail closed (Conftest evaluates all policies; Self-Audit reports the gap). (3) Integrity rule violation → build fails; engineer resolves or files an exception. (4) Performance degradation (query latency > p99 target) → investigate; consider re-indexing or scaling the cluster.
Runtime Verification	Prometheus metric neo4j_query_duration_seconds (histogram by query pattern); neo4j_integrity_rule_total (counter by rule, PASS/FAIL); neo4j_node_count and neo4j_edge_count (gauges); nightly self-audit (every active provision has a node, every active node has at least one edge, every integrity rule PASSes).

5.9 Graph Builders (Code, Constitution, CI)

The three graph builders are Python programs that run in CI on every commit and every release, populating the ATG with nodes and edges. The Constitution Builder produces Requirement nodes from the MRC; the Code Builder produces Architecture, ADR, Database, API, Code, Storage nodes from AST analysis and annotation parsing; the CI Builder produces Test, Evidence, Certification, Release nodes from CI metadata. The builders are what make the ATG live rather than hand-maintained: the edges are byproducts of normal development, not separate documentation labor.

Aspect	Specification
Purpose	Build and update the ATG automatically from the MRC, the codebase, and CI metadata, eliminating hand-maintained traceability.
Responsibilities	Constitution Builder: parse the MRC, produce Requirement nodes (one per provision). Code Builder:

	parse the codebase (AST + OpenAPI + catalog-info.yaml), produce Architecture/ADR/Database/API/Code/Storage nodes and implemented-by/exposed-by/backed-by/stored-in edges. CI Builder: parse CI run results, produce Test/Evidence/Certification/Release nodes and verified-by/produces/certified-by/released-in edges.
Inputs	Constitution Builder: the MRC. Code Builder: source code, OpenAPI specs, catalog-info.yaml, ADR Markdown files. CI Builder: GitHub Actions run results, test reports (JUnit XML), evidence files (links), release metadata.
Outputs	Graph mutations (Cypher CREATE/MERGE statements) applied to Neo4j; integrity rule reports; build summaries (node/edge count deltas).
Dependencies	Python 3.11+; libCST or tree-sitter (AST analysis); openapi-spec-validator (OpenAPI parsing); PyYAML (MRC and catalog-info); neo4j Python driver; the MRC; the codebase; the CI metadata.
Interfaces	CI job (triggered on commit / release); reads source from GitHub Actions workspace; writes mutations to Neo4j over Bolt; writes build summary to GitHub Check UI.
Deployment	GitHub Actions workflow (Ubuntu runner), runs on every commit to main and every PR. CI Builder also runs on every release.
Failure Modes	(1) Builder crashes (e.g., AST parse error on malformed code) → build fails; engineer fixes code or builder. (2) Neo4j unavailable → builder retries with exponential backoff; after 5 retries, build fails; engineer resolves. (3) Builder produces low-confidence edge (inferred) → edge marked low-confidence; nightly self-audit flags for human review. (4) Builder misses an edge that should exist → integrity rule IR-07 (orphan node) detects; engineer adds the missing annotation or fixes the builder.
Runtime Verification	CI build success (must pass); build summary (node count delta, edge count delta, integrity rule PASS/FAIL); nightly self-audit (every active code module has a Code node, every active API has an API node and an exposed-by edge, every active test has a

	verified-by edge).

5.10 OpenTelemetry Collector

The OpenTelemetry Collector is the vendor-neutral telemetry pipeline that receives metrics, logs, and traces from every application service and control-plane component, processes them (filter, sample, enrich), and exports them to the storage backends (Prometheus, Loki, Tempo). The Collector is the single telemetry ingress point; every service emits to the Collector, and the Collector routes to the appropriate backend.

Aspect	Specification
Purpose	Be the single telemetry ingress point for RC2, receiving metrics, logs, and traces from every component and routing them to storage backends.
Responsibilities	Receive telemetry over OTLP (gRPC and HTTP); process (filter, sample, enrich with resource attributes); export to Prometheus (metrics), Loki (logs), Tempo (traces); enforce retention policies; emit its own health metrics.
Inputs	OTLP telemetry from every service (the OpenTelemetry SDK in each service emits to the Collector); its own pipeline configuration (YAML).
Outputs	Metrics to Prometheus (via remote_write); logs to Loki (via lokiexporter); traces to Tempo (via otlpexporter); its own health metrics (scraped by Prometheus).
Dependencies	OpenTelemetry Collector (CNCF); Prometheus, Loki, Tempo (backends); the OpenTelemetry SDK in every service; the Kubernetes service mesh (for mTLS between service and Collector, if a service mesh is deployed).
Interfaces	OTLP receiver (gRPC :4317, HTTP :4318); Prometheus remote_write (to Prometheus); Loki push API; Tempo OTLP receiver; Prometheus scrape endpoint (its own /metrics).
Deployment	Kubernetes DaemonSet (one Collector pod per node) in the otel-system namespace; receives telemetry from services on the same node (node-local affinity). HA via the DaemonSet; no leader election needed.
Failure Modes	(1) Collector pod fails → Kubernetes restarts it; telemetry from services on that node is lost for the

	restart window (< 30 s). (2) Backend unavailable (e.g., Prometheus down) → Collector buffers to local disk (up to 1 GB); after buffer full, drops oldest; alert on buffer fill > 80%. (3) Collector config invalid → pod fails to start; alert immediately. (4) Telemetry volume exceeds capacity → Collector drops spans/logs (sampled); alert on drop rate > 0.
Runtime Verification	Prometheus metric otelcol_receiver_accepted_spans / otelcol_receiver_refused_spans (counters); otelcol_exporter_send_failed_spans (counter, alert if > 0); otelcol_processor_dropped_spans (counter, alert if > 0); nightly self-audit (every active service emits telemetry, every telemetry-emitting service has a recent telemetry signal in the backend).

5.11 Prometheus + Grafana

Prometheus is the metrics backend; Grafana is the visualization layer. Together they are the operational observability surface of RC2: Prometheus scrapes metrics from every service and from the OpenTelemetry Collector, evaluates alert rules, and feeds Grafana dashboards. Grafana surfaces the Constitutional Health Dashboard, the SLO dashboard, the operational dashboards, and the alert incidents.

Aspect	Specification
Purpose	Be the metrics backend (Prometheus) and the visualization layer (Grafana) for RC2 operational observability and constitutional health.
Responsibilities	Prometheus: scrape metrics from every service (15 s interval), evaluate alert rules, expose the Alertmanager API, retain metrics for 90 days (15 s resolution) and 1 year (1 min resolution via downsampling). Grafana: render dashboards (Constitutional Health, SLO, Operational), manage alert notification channels, provide ad-hoc query.
Inputs	Prometheus: scrape targets (every service's /metrics endpoint); alert rules (YAML, from the infrastructure repo); recording rules (pre-computed aggregates). Grafana: dashboard JSON (from the infrastructure repo); datasource config (Prometheus, Loki, Tempo, Neo4j).

Outputs	Prometheus: time-series database (TSDB); alert firing state (to Alertmanager); recording rule results. Grafana: rendered dashboards (HTTP); alert notifications (Slack, PagerDuty).
Dependencies	Prometheus 2.50+ (CNCF Graduated); Grafana 10+ (OSS); Alertmanager; the OpenTelemetry Collector (which exports to Prometheus); the infrastructure repo (alert rules, dashboards).
Interfaces	Prometheus: scrape HTTP (targets); PromQL HTTP API (queries); remote_write / remote_read (for long-term storage in Thanos or Mimir). Grafana: HTTP UI; Grafana HTTP API (dashboard CRUD); datasource proxies.
Deployment	Prometheus: StatefulSet in the monitoring namespace, 2 replicas (HA via Thanos or Mimir for federation), persistent volume (500 GB SSS). Grafana: Deployment in the monitoring namespace, 2 replicas, HA (shared dashboard storage in Postgres or MySQL). Alertmanager: StatefulSet, 3 replicas.
Failure Modes	(1) Prometheus unavailable → alerting stops; Alertmanager continues from last state; alert immediately. (2) Grafana unavailable → dashboards unavailable; alerting continues; alert. (3) TSDB full → Prometheus stops ingesting; alert at 80% full. (4) Recording rule slow → alert on rule evaluation duration > 5 s. (5) Alert storm (many alerts fire at once) → Alertmanager groups and inhibits; on-call engineer consults the runbook.
Runtime Verification	Prometheus self-metric prometheus_tsdb_head_series (gauge, alert if > 1 M); prometheus_rule_evaluation_duration_seconds (histogram); grafana_http_request_total (counter); nightly self-audit (every active service has a scrape target, every active alert rule has fired at least once in the last 30 days or has a documented reason for not firing).

5.12 Backstage Developer Portal

Backstage is Spotify's open-source developer portal framework, a CNCF project. In RC2, Backstage is the developer-facing surface of the governance dashboard: the catalog of services, the scorecards that grade

each service's compliance, the scaffolder that creates new compliant services, and the documentation browser. Backstage is the portal the engineer consults to see their service's compliance posture and to act on it.

Aspect	Specification
Purpose	Be the developer-facing surface of the governance dashboard: catalog, scorecards, scaffolder, docs.
Responsibilities	Catalog: index every service from its catalog-info.yaml; Scorecards: grade each service on pillars (Has Catalog Info, Has SLOs, Has Traces, Has Recent Evidence, Passes All GATE Criteria); Scaffolder: generate new compliant services from templates; Docs: render the MRC-derived docs.
Inputs	Catalog: catalog-info.yaml from every service repo (discovered via GitHub org ingestion). Scorecards: ATG queries (Cypher); OSCAL-AR feed. Scaffolder: templates (in the scaffolder repo). Docs: the MRC-derived docs.
Outputs	Web UI (catalog, scorecards, scaffolder, docs); HTTP API (for programmatic access); scorecard grades (consumed by Grafana for the Constitutional Health Dashboard).
Dependencies	Backstage 1.20+ (CNCF); PostgreSQL (catalog storage); the ATG (Neo4j, for scorecard data); the OSCAL-AR feed; the MRC-derived docs.
Interfaces	Web UI (HTTP); Backstage HTTP API (catalog, scorecards, scaffolder); GitHub (catalog ingestion via GitHub integration); Neo4j (scorecard queries via Cypher).
Deployment	Kubernetes Deployment in the developer-portal namespace, 2 replicas, HA. PostgreSQL for catalog storage (managed). Ingress with OIDC authentication (SSO via the Identity subsystem).
Failure Modes	(1) Backstage unavailable → developers cannot browse the catalog; alert. (2) Catalog ingestion fails (GitHub API rate limit) → catalog stale; alert; ingestion retries with backoff. (3) Scorecard query fails (Neo4j unavailable) → scorecards show stale; alert. (4) Scaffolder template fails → engineer cannot create new service; alert; engineer files a ticket.
Runtime Verification	Prometheus metric backstage_http_request_total

	(counter by route); backstage_catalog_entity_count (gauge); backstage_scorecard_query_duration_seconds (histogram); nightly self-audit (every active service has a catalog entity, every catalog entity has a scorecard with at least 4 of 5 pillars graded).

5.13 Sigstore (Cosign + Rekor + Fulcio)

Sigstore is the OpenSSF code-signing stack: Cosign signs artifacts (container images, binaries, blobs), Fulcio issues short-lived signing certificates (bound to an OIDC identity, e.g., the GitHub Actions workflow that ran), and Rekor is the transparency log (an append-only Merkle tree of every signature, publicly auditable). In RC2, Sigstore is the supply-chain integrity layer: every production artifact is signed with Cosign, the signature is recorded in Rekor, and the artifact is verified at deploy time by Flux.

Aspect	Specification
Purpose	Provide supply-chain integrity for RC2: sign every production artifact, record signatures in a transparency log, verify signatures at deploy time.
Responsibilities	Cosign: sign container images and blobs with OIDC-bound Fulcio certificates. Fulcio: issue short-lived signing certificates bound to the OIDC identity of the signer. Rekor: record every signature in an append-only transparency log; serve inclusion proofs.
Inputs	Cosign: container images (from the registry), blobs (SBOMs, OSCAL-AR fragments), OIDC tokens (from GitHub Actions OIDC provider). Fulcio: OIDC token (validated, identity extracted). Rekor: signature, certificate, artifact hash.
Outputs	Cosign: signatures (pushed to the registry alongside the image); attestations (e.g., SLSA L3 provenance, SBOM). Fulcio: short-lived signing certificate. Rekor: transparency log entry with inclusion proof.
Dependencies	Cosign 2.x; Fulcio (Sigstore project; the public-good Fulcio or a self-hosted Fulcio for air-gapped deployments); Rekor (public-good or self-hosted); the container registry; GitHub OIDC (for the signer identity).
Interfaces	Cosign CLI (in CI); registry API (push signatures); Fulcio REST API (issue certificate); Rekor REST API (log entry,

	inclusion proof).
Deployment	Cosign: CI step in GitHub Actions. Fulcio and Rekor: use the Sigstore public-good instance for non-air-gapped deployments; self-host Fulcio and Rekor in the control-plane namespace for air-gapped deployments (documented in ADR-011).
Failure Modes	(1) Fulcio unavailable → Cosign cannot obtain a signing certificate; CI fails; alert. (2) Rekor unavailable → Cosign cannot record the signature; CI fails (or, with --insecure-ignore-rekor, signs without log — used only in break-glass); alert. (3) Public-good Rekor outage → RC2 verification fails for artifacts signed during the outage; alert; engineer verifies manually or waits for Rekor recovery. (4) OIDC trust broken → Fulcio rejects the token; CI fails; alert immediately.
Runtime Verification	Prometheus metric sigstore_sign_total (counter by artifact type, success/failure); sigstore_verify_total (counter, success/failure); rekor_log_entries_total (counter); nightly self-audit (every production artifact deployed in the last 24 h has a Sigstore signature verifiable in Rekor, every signature's OIDC identity matches a permitted GitHub Actions workflow).

5.14 OSCAL Emitter and Repository

The OSCAL Emitter is the component that produces NIST OSCAL Assessment Results (OSCAL-AR) documents from CI run results and the MRC's certification criteria. The OSCAL Repository is the Git repository (opus-publica/certification) that stores every OSCAL-AR document (per-PR, nightly, per-release), indexed by the ATG, surfaced on Grafana and Backstage, and made available to external auditors via a signed feed. OSCAL is the lingua franca of certification, legible to FedRAMP assessors, funder-compliance officers, and indexer-eligibility reviewers.

Aspect	Specification
Purpose	Produce, store, and serve OSCAL Assessment Results documents that record the certification status of every criterion at every cadence.
Responsibilities	Emitter: read CI run results and MRC criteria; produce OSCAL-AR JSON conforming to the NIST OSCAL Assessment Results model; validate with oscal-cli. Repository: store OSCAL-AR documents (per-PR,

	nightly, per-release); index by ATG (Certification nodes, certified-by edges); serve signed feed to external auditors.
Inputs	Emitter: CI run results (GitHub Actions API); MRC criteria (certification/criteria/); the ATG Evidence nodes. Repository: OSCAL-AR documents (committed by the emitter).
Outputs	Emitter: OSCAL-AR JSON (per-PR fragment, nightly aggregate, per-release). Repository: Git history of OSCAL-AR documents; signed feed (JSON, signed with Cosign); ATG Certification nodes and certified-by edges.
Dependencies	oscal-cli (NIST OSCAL tooling); the MRC; the GitHub Actions API; the ATG (Neo4j); Cosign (feed signing); the certification repository.
Interfaces	Emitter: CI job (triggered on PR / nightly / release); reads CI results and MRC; writes OSCAL-AR to the certification repo. Repository: Git over HTTPS (clone, fetch); signed feed HTTPS endpoint (signed JSON, consumed by external auditors).
Deployment	Emitter: GitHub Actions workflow (Ubuntu runner). Repository: GitHub repository (managed). Feed server: a small HTTPS service in the control-plane namespace, serving the signed feed.
Failure Modes	(1) Emitter fails to produce valid OSCAL → CI fails; oscal-cli validation error reported; engineer fixes emitter or input. (2) Repository unavailable (GitHub outage) → emitter cannot commit; buffered locally; retried on recovery. (3) Feed server unavailable → external auditors cannot fetch the feed; alert; auditors fall back to repository clone. (4) Cosign feed signing fails → feed not updated; auditors see stale feed; alert.
Runtime Verification	Prometheus metric oscal_emit_total (counter by cadence, success/failure); oscal_feed_request_total (counter by consumer); nightly self-audit (every active criterion has an OSCAL-AR result in the last 24 h, every OSCAL-AR document validates against the OSCAL schema, every deployed release has a per-release OSCAL-AR).

5.15 Self-Audit Engine

The Self-Audit Engine (SAE) is the operational form of continuous assurance. It runs at two cadences: nightly (a GitHub Actions workflow at 02:00 UTC) and continuous (real-time rule-based and ML-based anomaly detection on the audit log). The SAE detects orphan artifacts, broken traceability edges, stale evidence, drift, vulnerabilities, expired credentials, and audit-log anomalies; it classifies findings SEV-1/2/3 and pages or tickets accordingly.

Aspect	Specification
Purpose	Realize continuous assurance (ISACA) and ISCM (NIST SP 800-137) by continuously auditing the RC2 platform for governance defects.
Responsibilities	Nightly: run every self-audit check (ATG queries, OSCAL-AR queries, driftctl, CycloneDX vulnerability scan, secret expiry); produce nightly audit report; commit to certification repo; summarize on Grafana; page on SEV-1. Continuous: evaluate rule-based Rego policies on every audit log record; evaluate ML-based anomaly detection on a 15-minute window; page on rule SEV-1/2; ticket on ML anomaly.
Inputs	Nightly: ATG (Cypher queries); OSCAL-AR (query); cloud provider API (driftctl); CycloneDX SBOM + vulnerability DB; secret manager API. Continuous: audit log stream (Kafka, the Audit subsystem's topic); rule policies (Rego); ML models (Isolation Forest, Local Outlier Factor).
Outputs	Nightly: audit report (JSON + PDF); Grafana summary; SEV-1 pages (PagerDuty); tickets (Jira) for SEV-2/3. Continuous: rule SEV-1/2 pages; ML anomaly tickets; ATG Evidence nodes (one per finding).
Dependencies	GitHub Actions (nightly); the ATG (Neo4j); the OSCAL-AR repository; driftctl; CycloneDX; the secret manager; Kafka (audit log stream); scikit-learn (ML); Rego / OPA (rule policies).
Interfaces	Nightly: GitHub Actions workflow (cron); reads from ATG, OSCAL-AR, cloud APIs; writes to certification repo, Grafana, PagerDuty, Jira. Continuous: Kafka consumer (audit log stream); evaluates Rego in OPA; evaluates ML model in a Python service; writes to PagerDuty, Jira, ATG.
Deployment	Nightly: GitHub Actions workflow. Continuous: a

	Kubernetes Deployment in the control-plane namespace, 2 replicas (HA); consumes from Kafka; Python service with scikit-learn; OPA sidecar for Rego evaluation.
Failure Modes	(1) Nightly workflow fails → no nightly audit report; alert; engineer reruns. (2) ATG unavailable during nightly → ATG checks skipped, reported in the audit report; alert. (3) Continuous SAE pod fails → Kubernetes restarts it; anomaly detection gap < 30 s; alert. (4) ML model degrades (high false-positive rate) → alert; retrain. (5) Kafka lag (continuous SAE cannot keep up) → alert; scale up.
Runtime Verification	Prometheus metric self_audit_nightly_run_total (counter by result); self_audit_continuous_findings_total (counter by severity, rule/ML); self_audit_kafka_lag (gauge, alert if > 1000); nightly self-audit (the SAE itself has run nightly for the last 7 days, every expected check ran, the continuous SAE has been up for > 99% of the last 24 h).

5.16 Anomaly Detection (Rule + ML)

The Anomaly Detection component is the continuous layer of the Self-Audit Engine, focused exclusively on detecting anomalies in the audit log. It has two layers: rule-based (Rego policies evaluated on every audit log record, deterministic and auditable) and ML-based (unsupervised models that detect novel patterns, probabilistic and explainable). The two layers are complementary: rules catch the known; ML catches the novel; together they provide defense-in-depth.

Aspect	Specification
Purpose	Detect anomalies in the audit log — known-bad patterns (rules) and novel deviations from normal behaviour (ML) — in real time.
Responsibilities	Rule layer: evaluate Rego policies on every audit log record; on match, page (SEV-1/2) or ticket (SEV-3). ML layer: train Isolation Forest and Local Outlier Factor models on the audit log history; score every record; on anomaly (score above threshold), ticket for human review. Both layers: emit ATG Evidence nodes; update the Grafana Anomalies panel.
Inputs	Audit log stream (Kafka, the Audit subsystem's topic);

	rule policies (Rego, in the MRC); ML models (Isolation Forest, Local Outlier Factor, trained on 90 days of audit log history); feature engineering config (which fields to extract, how to encode).
Outputs	Rule findings (SEV-1/2 pages, SEV-3 tickets, ATG Evidence nodes); ML anomaly tickets (with anomaly score, contributing features, and the audit log record for human review); Grafana Anomalies panel updates.
Dependencies	Kafka (audit log stream); OPA (rule evaluation); scikit-learn (ML); the MRC (rule policies); the ATG (Neo4j, for Evidence nodes); PagerDuty (pages); Jira (tickets); Grafana (panel updates).
Interfaces	Kafka consumer (audit log stream); OPA REST API (rule evaluation); scikit-learn in-process (ML scoring); PagerDuty API (pages); Jira API (tickets); Grafana HTTP API (panel updates).
Deployment	Kubernetes Deployment in the control-plane namespace, 2 replicas (HA); consumes from Kafka; Python service with scikit-learn; OPA sidecar for Rego evaluation. ML models retrained nightly; rule policies updated via MRC PR.
Failure Modes	(1) Rule policy crashes (Rego error) → rule evaluation fails; alert; engineer fixes policy. (2) ML model not trained (first deployment) → ML layer disabled; rule layer remains active; alert; train model on available history. (3) ML model degrades (high false-positive rate) → alert; retrain; if persistent, lower the anomaly threshold or disable ML layer (rule layer remains). (4) Kafka lag → alert; scale up.
Runtime Verification	Prometheus metric anomaly_rule_findings_total (counter by severity); anomaly_ml_findings_total (counter, with score distribution); anomaly_rule_evaluation_duration_seconds (histogram); anomaly_ml_inference_duration_seconds (histogram); nightly self-audit (the rule layer evaluated every audit log record in the last 24 h, the ML layer scored every record, every SEV-1/2 finding was paged within 5 min).

The rule layer is the primary auditor-trustable signal: deterministic, auditable, explainable. The ML layer is the early-warning radar: probabilistic, novel, fallible. The two-layer design is deliberate; a single-layer

design (rules only) misses novel attacks, and a single-layer design (ML only) produces false positives that erode trust. Together, they cover both the known and the novel.

Chapter 6 — Data Flow Architecture

This chapter specifies the principal data flows through the RC2 architecture. The flows are presented as numbered step sequences with diagrams; each step identifies the component that performs it, the inputs it consumes, and the outputs it produces. The flows are normative: an implementation whose flow deviates from the specified flow in a way that bypasses a control-plane enforcement point is defective.

6.1 Pull Request Flow

The Pull Request flow is the most-frequently-executed flow in RC2 (every PR). It exercises the full pre-merge pipeline: Conftest gate, ATG build, document regeneration, OSCAL-AR fragment emission, Backstage scorecard update, reviewer approval, merge. The flow is the operational realization of the shift-left principle (verify at PR time, not release time).

```
Engineer opens PR (modifies services/doi-minter/ + /contracts/SUB-10.yaml)
                                    |
                                    v
                [1] GitHub triggers pre-merge workflow
                                    |
                                    v
[2] Conftest: ATG touch-query -> which provisions does this PR touch?
                                    |
                                    v
[3] Conftest: evaluate relevant Rego policies -> PASS / FAIL per policy
                                    |
                                    v
 [4] Code Builder: update ATG with new/modified nodes + edges
                                    |
                                    v
 [5] Integrity Verifier: run 8 integrity rules -> block if any FAIL
                                    |
                                    v
  [6] Document Generators: regenerate affected .docx/.pdf documents
                                    |
                                    v
[7] CI: run service test suite -> produce evidence (logs, coverage)
                                    |
                                    v
        [8] CI Builder: link Evidence nodes to ATG
                                    |
                                    v
[9] OSCAL Emitter: emit per-PR OSCAL-AR fragment -> certification repo
                                    |
                                    v
    [10] Backstage Scorecard for DOI Minter service updates
```

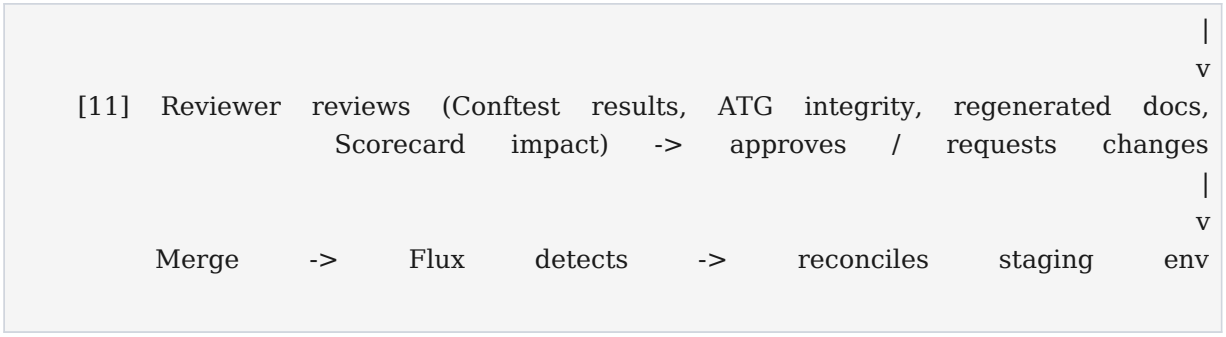


Figure 6.1: Pull Request Flow (11 steps)

The flow's design enforces the shift-left principle: every governance check that can run at PR time does run at PR time, blocking the merge rather than the release. A PR that passes the flow is one that the Constitution (as encoded in the MRC) approves; the reviewer's role is judgement (does this change make sense?), not verification (does this change satisfy the provisions?). The flow is also the source of the per-PR OSCAL-AR fragment, which becomes the PR's certification preview and, on merge, part of the release's certification evidence.

6.2 Publication Event Flow

The Publication Event flow is the runtime flow that executes when an article is published. It exercises the data plane and its external integrations: the Publication subsystem emits an ArticlePublished event, which triggers DOI minting, ORCID sync, preservation deposit, search indexing, citation graph update, and audit log recording. The flow is the operational realization of the publication state machine (Constitution Volume I Chapter 11).

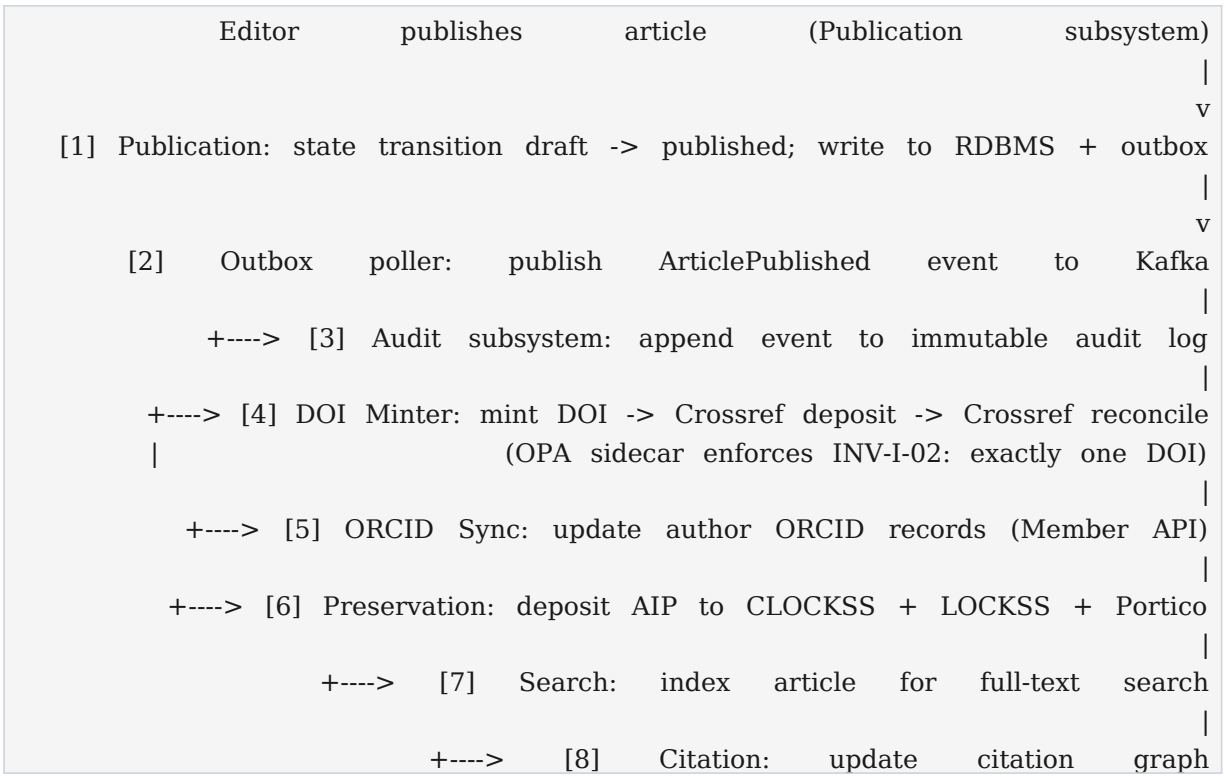


Figure 6.2: Publication Event Flow (11 steps, event-driven fan-out)

The flow's design uses the outbox pattern to guarantee atomicity: the publication and the event publication are in the same RDBMS transaction; if the transaction commits, the event is published; if it rolls back, neither occurs. The fan-out to downstream consumers is asynchronous, with each consumer responsible for its own retry, idempotency, and reconciliation. The OPA sidecar in the DOI Minter enforces INV-I-02 (exactly one DOI per published article version) at runtime, providing defense-in-depth against a bug that bypasses the pre-merge Conftest gate. Every step emits OpenTelemetry traces, which become evidence in the ATG and which the self-audit can use to verify that the publication completed correctly.

6.3 Certification Cycle Flow

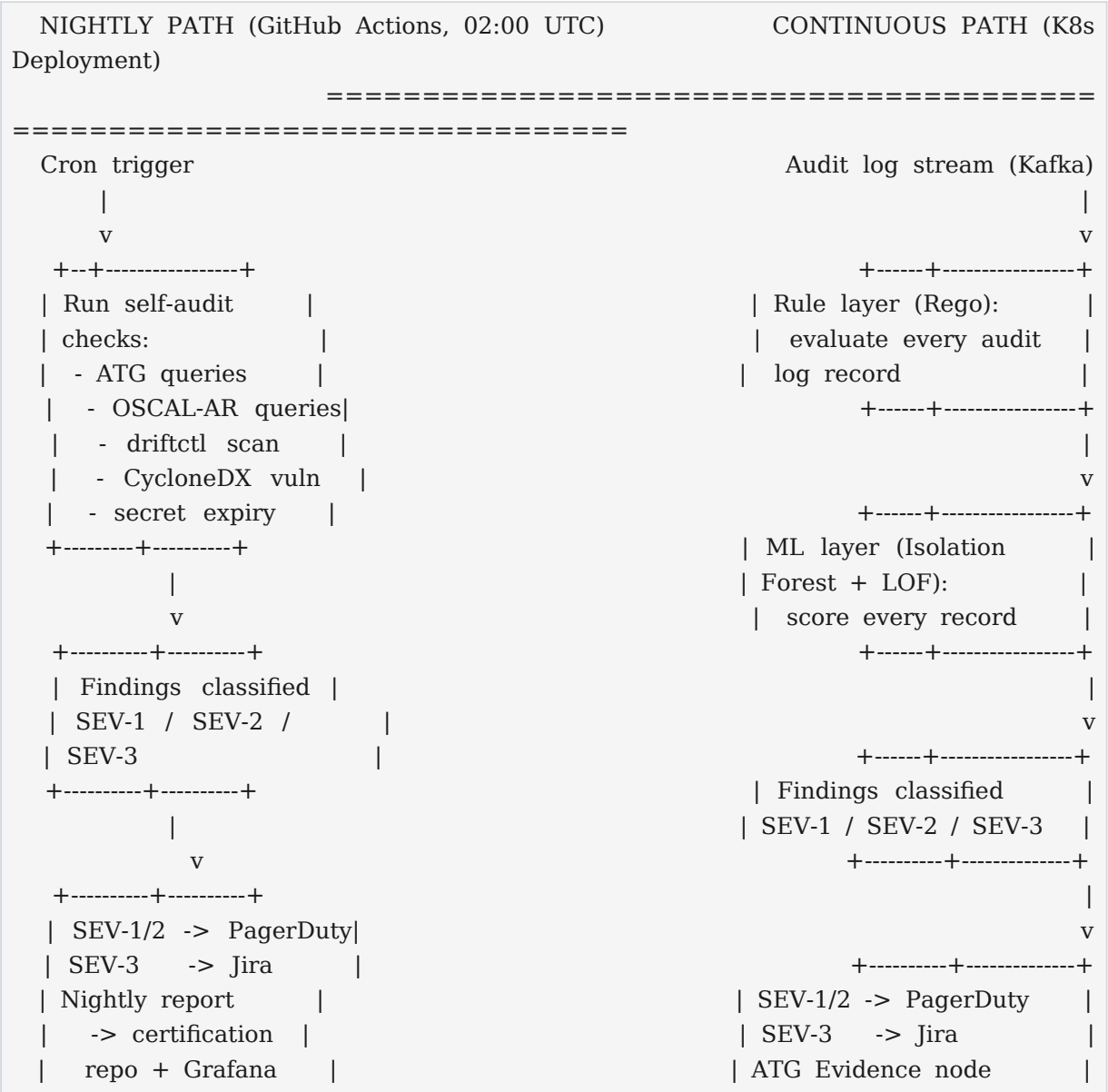
The Certification Cycle flow is the flow by which every certification criterion is evaluated at three cadences: per-PR, nightly, and per-release. The flow is the operational realization of the Continuous Certification Engine. The three cadences together provide continuous assurance: per-PR catches issues at PR time; nightly catches runtime issues; per-release gates the production deployment.

Cadence	Trigger	Criteria Evaluated	Output
Per-PR	PR opened or updated.	Every Rego policy whose provisions the PR touches (ATG touch-query).	GitHub Check (PASS/FAIL); per-PR OSCAL-AR fragment.
Nightly	Cron (02:00 UTC).	Every criterion with a runtime component (invariant checks, drift checks, vulnerability scans, audit-log anomaly detection).	Nightly OSCAL-AR; Grafana summary; SEV-1 pages.
Per-release	Release tag created.	Every GATE criterion (criterion marked gate: true in the MRC).	Release OSCAL-AR; RCA Environment approval gate.

The per-PR cadence is the shift-left layer; it catches defects before they merge. The nightly cadence is the runtime-assurance layer; it catches defects that emerge in the running system (drift, vulnerabilities, anomalies). The per-release cadence is the production gate; it certifies that the release as a whole satisfies every GATE criterion before deployment. The three cadences compose: a per-PR PASS contributes to the per-release PASS; a nightly SEV-1 finding blocks the next release. The OSCAL-AR documents from all three cadences are the evidence of continuous assurance, available to internal and external auditors.

6.4 Self-Audit Flow

The Self-Audit flow is the flow by which the Self-Audit Engine (SAE) continuously audits the RC2 platform. The flow has two parallel paths: the nightly path (a GitHub Actions workflow) and the continuous path (a Kubernetes Deployment consuming the audit log stream). Both paths produce findings classified SEV-1/2/3, with SEV-1/2 paging the on-call engineer and SEV-3 opening a ticket.



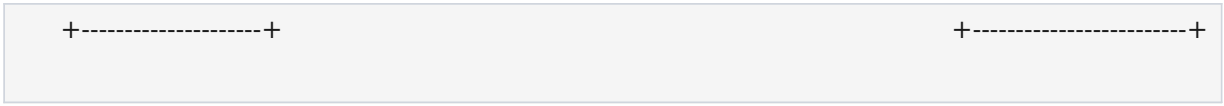


Figure 6.3: Self-Audit Flow (nightly + continuous, parallel paths)

The nightly path catches the defects that require a full scan (orphan nodes, stale evidence, drift, vulnerabilities); the continuous path catches the defects that require real-time detection (audit-log anomalies). The two paths are complementary: the nightly path is exhaustive but slow (runs once); the continuous path is selective but fast (runs in milliseconds per record). Together they provide defense-in-depth for the audit log and the platform's governance state. The weekly CSA review consults both paths' outputs: the nightly reports from the past week, and the continuous findings aggregated into the Grafana Anomalies panel.

6.5 Incident Response Flow

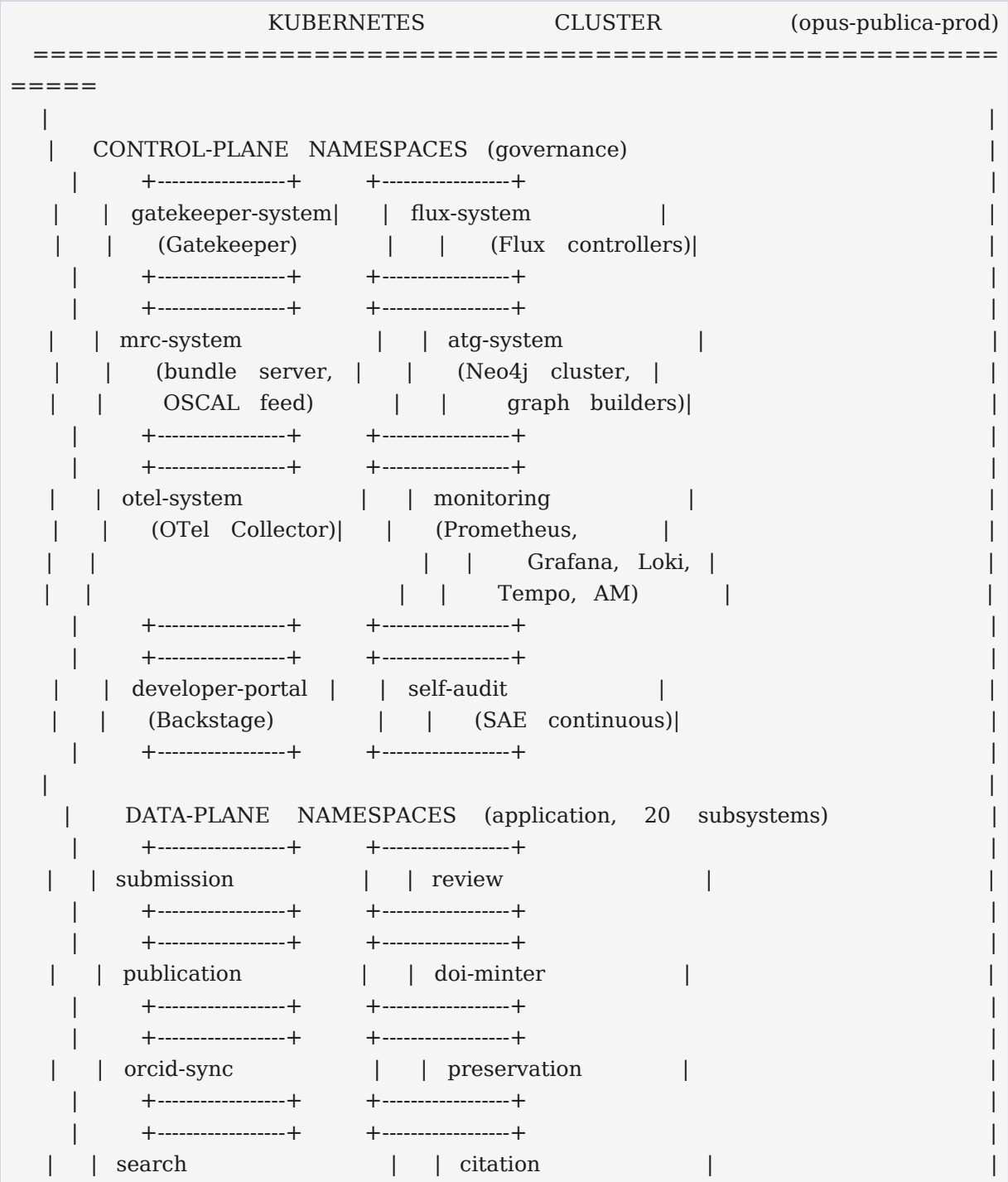
The Incident Response flow is the flow by which an RC2 incident (a SEV-1 finding, an alert firing, a user-reported outage) is detected, triaged, mitigated, resolved, and post-mortemed. The flow is governed by the RC1 Playbook (Chapter 20: Incident Response); RC2 automates the detection, paging, and evidence-collection steps. The flow is the operationalization of the platform's RTO/RPO NFRs (Section 2.4).

1. **Step 1.** Detection: Prometheus alert fires (SLO burn, invariant violation, anomaly) OR a SEV-1 self-audit finding pages OR a user reports an outage.
2. **Step 2.** Paging: PagerDuty pages the on-call engineer; the page includes the alert name, severity, the relevant ATG query (for context), and a link to the runbook.
3. **Step 3.** Triage: on-call acknowledges within 5 min (SEV-1) or 15 min (SEV-2); classifies the incident; declares a major incident if scope warrants.
4. **Step 4.** Mitigation: on-call executes the runbook; common mitigations include rollback (Flux redeploys previous image), feature flag off, or scale up. The mitigation is recorded in the incident channel.
5. **Step 5.** Evidence collection: the incident channel bot captures the ATG query results, the OpenTelemetry traces, the Grafana snapshots, and the relevant audit log entries, attaching them to the incident record.
6. **Step 6.** Resolution: on-call confirms the incident is resolved; closes the incident in PagerDuty; the incident record is preserved (immutable, 7-year retention).
7. **Step 7.** Post-mortem: within 5 business days, the on-call (now the incident owner) writes a blameless post-mortem; the post-mortem identifies the root cause, the contributing factors, the action items (with owners and due dates), and the ATG/Evidence links.
8. **Step 8.** Action item follow-through: action items are tracked in Jira; the weekly CSA review confirms progress; an action item overdue > 30 days escalates to the CSA.

Chapter 7 — Deployment Topology

7.1 Kubernetes Namespace Layout

The RC2 platform is deployed on Kubernetes. The Kubernetes cluster is partitioned into namespaces that reflect the control-plane / data-plane separation (Section 2.3). The namespace layout is normative: an implementation that deploys a data-plane service into a control-plane namespace (or vice versa) is defective.



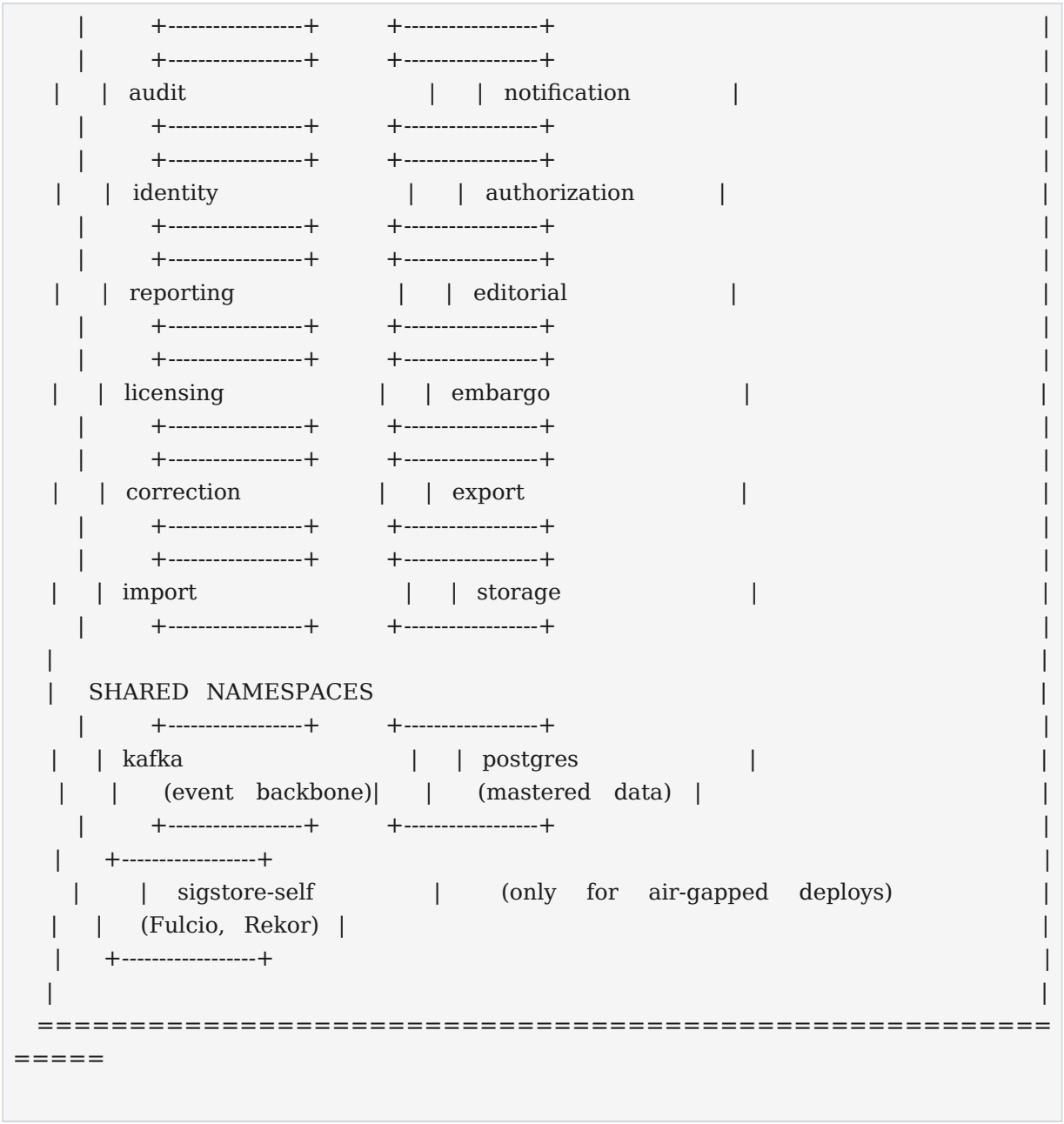


Figure 7.1: Kubernetes Namespace Layout (control plane, data plane, shared)

7.2 Environment Strategy

RC2 has four environments: dev, staging, prod, and prod-equivalent (the certification environment). The environments are deployed as separate Kubernetes clusters (prod, prod-equivalent) or as separate namespaces within a shared cluster (dev, staging). Each environment has its own instance of every control-plane and data-plane component; the MRC repository and the application code repository are shared (single source of truth), but the deployed state is per-environment.

Environment	Purpose	Cluster	Data
dev	Engineer local dev; per-	Shared dev cluster.	Synthetic; resettable.

	engineer or per-team namespace.		
staging	Pre-production; integration testing; release rehearsal.	Shared staging cluster.	Production-shaped; anonymized sample.
prod-equivalent	Certification environment; exact replica of prod; the CCE's per-release gate runs here.	Dedicated prod-equivalent cluster.	Production-shaped; full anonymized copy of prod.
prod	Production; the live platform.	Dedicated prod cluster.	Live user data.

The prod-equivalent environment is the certification environment: it is an exact replica of prod (same Kubernetes version, same component versions, same infrastructure-as-code), populated with an anonymized copy of prod data, on which the per-release certification gate runs. The prod-equivalent environment exists because some certification criteria cannot be evaluated in staging (staging's data is too small) and cannot be evaluated in prod (evaluating in prod would risk the live system). The per-release flow is: deploy to prod-equivalent; run the per-release certification gate; if all GATE criteria PASS, deploy to prod. The prod-equivalent environment is destroyed and rebuilt for every release, ensuring it is always a clean replica.

7.3 GitOps Deployment Model

RC2 uses Flux (CNCF Graduated) as its GitOps controller. Every Kubernetes manifest lives in the infrastructure repository (opus-publica/infrastructure), organized by environment (directories: dev/, staging/, prod-equivalent/, prod/) and by component (subdirectories per namespace). Flux watches the repository; on a change, Flux reconciles the live cluster state to the declared state. There is no manual kubectl apply in RC2; every change is a PR to the infrastructure repo, reviewed, merged, and reconciled by Flux.

The sync cadence is configurable. For dev, Flux polls every 1 min (rapid iteration). For staging, every 5 min. For prod-equivalent and prod, every 5 min, but with a manual approval gate (a GitHub Environment approval) before Flux reconciles to prod. Image updates (a new container image pushed to the registry) are detected by Flux's image-automation-controller, which opens a PR to update the manifest's image tag; the PR is reviewed and merged like any other. The GitOps model provides a complete audit trail: every change to the live cluster is a Git commit, with the author, the reviewer, the timestamp, and the diff, all in the immutable Git history.

7.4 Secret Management

RC2 uses HashiCorp Vault (or a managed equivalent: AWS Secrets Manager, GCP Secret Manager, Azure Key Vault) for secret management. Secrets are never stored in Git (even encrypted); the infrastructure repo stores a reference to the secret (a Vault path or a cloud secret ARN), and the Flux ExternalSecret operator fetches the secret at deploy time and injects it as a Kubernetes Secret. This pattern keeps secrets out of Git history (which is immutable and hard to scrub) while keeping the GitOps model intact (the reference is in Git, the value is in the secret manager).

Secret rotation is automated. Database credentials rotate every 90 days; external API credentials (Crossref, ORCID, CLOCKSS) rotate per the provider's policy (typically annually); OIDC signing keys rotate every 30 days. The rotation is performed by the secret manager; the ExternalSecret operator detects the new value and redeploys the affected pods. Every secret access is logged by the secret manager and shipped to the audit log; the self-audit's continuous layer flags anomalous access (a secret accessed from an unusual geography, by an unusual workload, at an unusual time).

7.5 Network Policy

Kubernetes NetworkPolicy enforces the control-plane / data-plane separation at the network level. The default policy is DENY-ALL: a pod can communicate only with pods explicitly allowed by a NetworkPolicy. The policies are:

- **Data-plane isolation:** data-plane pods can communicate with data-plane pods (within and across data-plane namespaces) and with shared namespaces (kafka, postgres); data-plane pods CANNOT communicate with control-plane pods (except for the OpenTelemetry Collector, which receives telemetry from all pods).
- **Control-plane isolation:** control-plane pods can communicate with control-plane pods; control-plane pods can communicate with shared namespaces (kafka for the self-audit's audit log consumption, postgres for Backstage's catalog storage); control-plane pods CANNOT initiate communication with data-plane pods (the data plane does not accept inbound from the control plane except via the ATG query API, which is read-only and authenticated).
- **Egress controls:** egress to external systems (Crossref, ORCID, CLOCKSS, the public Sigstore Rekor) is allowed only from specific data-plane namespaces (doi-minter, orcid-sync, preservation) and specific control-plane namespaces (sigstore-self for self-hosted Sigstore). Egress to all other external hosts is denied.
- **Ingress controls:** ingress from the public internet is allowed only via the platform Ingress (HTTPS, OIDC-authenticated for editor/author facing services; mTLS for system-to-system). Direct pod access from the internet is denied.

Chapter 8 — Security Architecture

8.1 Threat Model (STRIDE per Component)

The RC2 security architecture is grounded in a STRIDE threat model (Microsoft; Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) applied per component. The threat model identifies the threats each component faces and the controls that mitigate them. The model is normative: an implementation that omits a control listed for a threat is defective.

Component	Top STRIDE Threats	Controls
MRC Repository	Tampering (provision modified without authorization); Repudiation (provision modified, author denies).	Branch protection (1 review, all CI passing); signed commits (GPG or Sigstore gitsign); immutable release tags; PR audit log.
OPA (all configs)	Tampering (policy modified to be permissive); DoS (policy evaluation exhausts resources).	Signed bundle (Cosign); bundle signature verification (fail closed); policy complexity lint (opa check); decision timeout (100 ms).
Neo4j ATG	Information Disclosure (graph data exfiltrated); Tampering (graph mutated outside the builders).	Network policy (only control-plane and authorized data-plane query service accounts); write-restricted Cypher (only builder service accounts can write); RBAC (read-only for dashboards).
GitHub Actions	Elevation of Privilege (malicious PR gains access to secrets); Spoofing (workflow impersonates another).	OIDC token (per-workload identity, no long-lived secrets); Environment approvals (manual gate for prod); secret scope (per-environment); pull_request_target avoidance (anti-pattern).
Flux	Tampering (manifest modified to deploy malicious image); Elevation of Privilege (Flux's service account abused).	Branch protection on infrastructure repo; image signature verification (Cosign + Fulcio); Gatekeeper admission control (even Flux's applies are policy-governed); least-privilege RBAC for Flux's service account.
Application Services	Spoofing (unauthorized caller); Tampering (data modified outside state machine); Repudiation (action taken, actor denies); Information	OIDC authentication; OPA sidecar authorization; state machine enforcement (no out-of-band mutation); audit log on every

	Disclosure (PII exfiltrated); DoS (request flood); EoP (privilege escalation).	action; PII field-level encryption; rate limiting; least-privilege RBAC.
External Integrations	Spoofing (caller impersonates Crossref); Tampering (response modified in transit); DoS (external service unavailable).	TLS 1.3 (mTLS where supported); API key rotation; idempotent retries with backoff; circuit breaker; reconciliation job detects divergence.
Audit Log	Tampering (log entry modified or deleted); Information Disclosure (audit log read by unauthorized); DoS (audit log flooded).	Append-only (no update/delete); S3 Object Lock (Compliance mode, 7-year retention); nightly hash chain (every entry's hash includes the previous entry's hash); RBAC (write-only for services, read-only for auditors); rate limiting per writer.
Sigstore	Tampering (signature forged); DoS (Rekor unavailable).	Fulcio short-lived certs (no long-lived signing keys); Rekor transparency log (publicly auditable); keyless signing (no key to steal); fallback to public-good Rekor if self-hosted fails.
Self-Audit Engine	Elevation of Privilege (SAE service account abused to read secrets); Tampering (SAE finding suppressed).	Least-privilege RBAC (SAE reads audit log, ATG, OSCAL-AR; does not read secrets); SAE findings committed to immutable certification repo (cannot be suppressed silently); SAE access logged.

8.2 Authentication

RC2 has three authentication domains: human (authors, editors, operators), workload (services, CI jobs, Flux), and external system (Crossref, ORCID, preservation services). Each domain has a distinct authentication mechanism, chosen to minimize long-lived credentials and maximize auditability.

- **Human:** OIDC (OpenID Connect) via the Identity subsystem. Human users authenticate to the Identity subsystem (which federates to the organization's IdP: Okta, Google Workspace, or Azure AD); the Identity subsystem issues an OIDC token, accepted by every service. Service-to-service calls within the data plane use mTLS (mutual TLS) for transport-level authentication, with the OIDC token for identity.

- **Workload:** GitHub OIDC for CI jobs (a workflow run receives an OIDC token bound to the workflow, the repository, and the ref; the token is exchanged for short-lived cloud credentials via the cloud provider's OIDC trust). Kubernetes ServiceAccounts for in-cluster workloads (a pod's ServiceAccount token is bound to the ServiceAccount and the namespace). No long-lived workload credentials exist in RC2.
- **External system:** TLS 1.3 for transport; API keys for application-level authentication (rotated per the provider's policy); mTLS where the provider supports it. Inbound from external systems (OAI-PMH, SWORD): OIDC where supported, API key otherwise, IP allowlist as a defense-in-depth.

8.3 Authorization

RC2 has two authorization mechanisms: Kubernetes RBAC (for control-plane operations) and OPA (for application-level authorization within the data plane). The two mechanisms are complementary: RBAC governs who can do what to Kubernetes objects (create a Deployment, read a Secret); OPA governs who can do what to application data (publish an article, mint a DOI, deposit to CLOCKSS).

Kubernetes RBAC is configured per namespace with least privilege. The Flux service account can manage (create, update, delete) resources in the data-plane and shared namespaces, but cannot manage resources in the control-plane namespaces (the control plane governs itself). The Gatekeeper service account can read resources in all namespaces (for audit) but cannot modify any. The self-audit service account can read resources in all namespaces (for drift detection) but cannot modify any. Human operators (via SSO) receive view access by default; edit access is granted per-namespace via group membership (e.g., the platform-ops group can edit the monitoring namespace).

OPA authorization is enforced by the sidecar in every application service. Before performing a policy-governed action, the service queries the sidecar with the action (verb and object), the actor (from the OIDC token), and the context (the article, the submission, the journal). OPA evaluates the relevant Rego policy and returns ALLOW or DENY with the matched policy ID. The decision is logged to the audit log and the ATG. A DENY is returned to the caller with the policy ID and the provision reference, so the caller can understand why the action was denied.

8.4 Supply Chain (SLSA L3, SBOM, Signing, Verification)

RC2's supply chain achieves SLSA (Supply-chain Levels for Software Artifacts) Level 3. SLSA L3 requires: (1) the build process runs on a hardened build service (GitHub Actions, with the workflow's parameters logged and verified); (2) the build provenance is recorded (the slsa-github-generator produces a provenance attestation for every artifact); (3) the provenance is verifiable (the attestation is signed with Sigstore and recorded in Rekor); (4) the provenance is verified at deploy time (Flux verifies the attestation before deploying the image).

Every artifact (container image, binary, OSCAL-AR document, MRC bundle) has a Software Bill of Materials (SBOM) generated by Syft in CycloneDX 1.6 format. The SBOM is signed with Cosign, recorded in Rekor, and stored alongside the artifact. The self-audit's nightly check queries the SBOM against a

vulnerability database (the OpenSSF OSV database) and flags any vulnerable dependency as a SEV-1 (critical CVE), SEV-2 (high CVE), or SEV-3 (medium/low CVE) finding. Every artifact is signed with Cosign (keyless, using Fulcio's OIDC-bound short-lived certificates); the signature is recorded in Rekor. At deploy time, Flux verifies the signature (and the SLSA L3 attestation) before deploying the image; a signature verification failure blocks the deployment.

8.5 Audit Trail

The audit trail is the immutable, tamper-evident record of every security-relevant action in RC2. The audit trail is the operational realization of the auditability invariant (INV-A-01 through INV-A-04 in Constitution Volume II Chapter 26). The audit trail is the single most important security artifact in RC2: it is the evidence that supports every certification decision and every incident investigation.

The audit log is stored in Loki (for short-term, 90-day, queryable access) and in S3 with Object Lock (Compliance mode, 7-year retention, for long-term, tamper-evident storage). Every audit log entry is structured JSON, conforming to a JSON Schema, with the following fields: timestamp (ISO 8601), actor (human OIDC subject or workload ServiceAccount), action (verb), object (the resource affected), result (success/failure), request_id (correlation ID), and hash (SHA-256 of the entry's content, including the previous entry's hash, forming a hash chain). The hash chain is verified nightly by the self-audit; a broken chain is a SEV-1 finding.

What is logged: every API call to every service (with the actor, the action, the object, the result); every OPA decision (with the policy ID, the input, the decision); every Kubernetes admission request (with the requester, the object, the Gatekeeper decision); every Git operation (commit, merge, tag); every secret access (with the workload, the secret path); every external integration call (with the destination, the result). Retention is 7 years (legal hold for scholarly record preservation requirements). Tamper detection is via the nightly hash chain verification and via S3 Object Lock (which prevents deletion even by an administrator within the retention period).

Chapter 9 — Observability Architecture

The RC2 observability architecture follows the OpenTelemetry model: every service emits metrics, logs, and traces via the OpenTelemetry SDK; the OpenTelemetry Collector receives, processes, and exports to the storage backends (Prometheus for metrics, Loki for logs, Tempo for traces); Grafana is the visualization layer. The architecture is vendor-neutral (any OTel-compatible backend can be substituted) and unified (a single correlation ID links a metric, a log entry, and a trace span, enabling end-to-end investigation).

9.1 Metrics

Metrics are time-series data: counters (e.g., opa_decision_total), gauges (e.g., neo4j_node_count), histograms (e.g., opa_decision_latency_seconds). Every service and every control-plane component exposes a /metrics endpoint in the OpenTelemetry format; the OpenTelemetry Collector scrapes (or receives via OTLP) and exports to Prometheus. Prometheus stores metrics for 90 days at 15 s resolution and 1 year at 1 min resolution (via Thanos or Mimir downsampling).

RC2 distinguishes three classes of metric: operational (latency, throughput, error rate, the standard RED metrics), governance (invariant status, certification coverage, ATG integrity, drift detected), and SLO (error budget consumed, burn rate, projected breach date). The governance metrics are the novel contribution of RC2: in RC1, governance status was a quarterly PDF report; in RC2, it is a real-time metric, visible on the Grafana Constitutional Health Dashboard, alerting on burn. Every governance metric is backed by an ATG query or an OSCAL-AR query, so the metric's value is traceable to the underlying evidence.

9.2 Logs

Logs are structured JSON records emitted by every service via the OpenTelemetry SDK's logging API. Structured logging (JSON, not free-form text) is mandatory: a log entry without a timestamp, level, message, and correlation ID is defective. Logs are shipped to the OpenTelemetry Collector, which exports to Loki (short-term, 90-day, queryable) and to S3 with Object Lock (long-term, 7-year, tamper-evident).

The audit log (Section 8.5) is a subset of the log stream: every audit log entry is also a log entry, but with additional fields (actor, action, object, hash). The audit log is stored separately (S3 Object Lock, append-only, hash-chained) to satisfy the immutability invariant. The operational log (non-audit entries) is stored in Loki for queryability and in S3 (without Object Lock) for cost-effective long-term retention. Log retention: operational logs 90 days in Loki, 1 year in S3; audit logs 90 days in Loki, 7 years in S3 with Object Lock.

9.3 Traces

Traces are distributed traces: a request's journey across services, broken into spans (one per service call). Every service is instrumented with the OpenTelemetry SDK, which automatically creates spans for

inbound HTTP calls and outbound HTTP/database calls; services add custom spans for business-meaningful operations (e.g., the DOI Minter adds a span for the Crossref deposit). Traces are sampled: 100% sampling for the publication path (the critical path), 10% for other paths, with dynamic sampling (a service under load can reduce its sampling rate to manage telemetry volume).

Traces are shipped to the OpenTelemetry Collector, which exports to Tempo (or Jaeger). Tempo stores traces for 30 days. The correlation ID (the trace ID) is propagated in every log entry and every metric label, so an engineer investigating an incident can move from a metric (e.g., a latency spike) to the relevant logs (filter by trace ID) to the relevant trace (the full call chain) in a single Grafana investigation. Traces are also evidence: the CI Builder creates ATG Evidence nodes from publication-path traces, so the publication's execution is recorded in the ATG.

9.4 Dashboards

RC2 has three categories of dashboard, each with a distinct audience and purpose. The dashboards are defined as JSON in the infrastructure repo and deployed to Grafana via Flux; dashboard changes are PRs, reviewed and merged like any other change.

- **Operator dashboards (Grafana):** the Constitutional Health Dashboard (the platform's compliance posture: invariant status, SLO burn, certification coverage, audit anomalies, supply-chain integrity, ATG integrity); the SLO Dashboard (the 10 SLOs, error budget consumed, projected breach); the Operational Dashboards (per-service RED metrics, per-component health, Kafka lag, database connections).
- **Developer dashboards (Backstage):** the service catalog (every service, with its owner, its scorecard, its recent incidents); the scorecards (per-service compliance grade, with pillars); the scaffolder (create a new compliant service); the docs browser (the MRC-derived documents).
- **Auditor dashboards (Grafana, read-only):** the audit feed dashboard (the per-release OSCAL-AR, the nightly audit report, the ATG integrity status); the supply-chain dashboard (the SLSA L3 verification status, the Rekor log entries).

9.5 Alerting

RC2 alerting is SLO-based (Google SRE Workbook) and invariant-based (RC2-specific). SLO-based alerts fire on burn rate: a fast-burn alert (2% error budget consumed in 1 hour) pages immediately; a slow-burn alert (10% error budget consumed in 6 hours) opens a ticket. Invariant-based alerts fire on invariant violation: a SEV-1 invariant violation (e.g., INV-I-02 violated: two DOIs for one published article version) pages immediately; the page includes the invariant ID, the violation detail, the ATG query (for context), and the runbook link.

Alert routing is via Alertmanager. SEV-1 alerts page the on-call engineer (PagerDuty); SEV-2 alerts open a high-priority ticket (Jira) and notify the team channel (Slack); SEV-3 alerts open a normal ticket. Alerts are deduplicated (the same alert firing repeatedly within a window is grouped) and inhibited (a SEV-1 alert inhibits the SEV-2 alerts that derive from it). The weekly CSA review audits the alert noise: an alert that

fires frequently without action is a candidate for tuning or suppression (with documented justification); an alert that fires rarely is a candidate for verification (is the alert still relevant?). Alert fatigue is treated as a defect: an engineer who ignores an alert is a symptom, not a cause; the cause is the alert that fires without actionable information.

Chapter 10 — Counterarguments and Limitations

This chapter addresses the limitations of the RC2 architecture and the counterarguments against it. The architecture is not a panacea; it introduces new dependencies, new failure modes, and new costs. A rigorous specification acknowledges these honestly and specifies how they are mitigated, rather than presenting the architecture as unalloyed improvement. The limitations are not reasons to abandon RC2; they are reasons to implement it carefully, with the mitigations specified.

10.1 Limitation: Architecture Complexity

The RC2 architecture has approximately 20 components (16 specified in Chapter 5, plus the application subsystems, the event backbone, and storage). Each component has its own deployment, its own failure modes, its own upgrade cadence, and its own operational surface. The complexity is real: an on-call engineer must understand the interactions of 20 components to debug an incident; a new engineer must learn 20 components to become productive. Complexity is the enemy of reliability: more components mean more failure modes, more interactions, more opportunities for misconfiguration.

The mitigation is three-fold. First, the components are loosely coupled: each component has a well-defined interface (Chapter 5), and the failure of one component degrades, rather than breaks, the system (e.g., Neo4j failure degrades the ATG but does not block publication). Second, the observability architecture (Chapter 9) is designed to make the complexity legible: a single Grafana dashboard shows the health of every component, and a single investigation can follow a request across components via the correlation ID. Third, the architecture is documented (this specification, plus the ADRs and the runbooks), so the knowledge is captured, not tribal. The complexity is the cost of the governance capability; the mitigation is the discipline of loose coupling, observability, and documentation.

10.2 Limitation: Single Points of Failure

Despite HA deployment for most components, the RC2 architecture has potential single points of failure: Neo4j (a causal cluster has one leader at a time; a leader failure triggers an election, but a cluster-wide failure is a single point), the OPA bundle server (a single service; failure means OPA instances serve stale bundles, and after 24 h, fail closed), and Sigstore Rekor (the transparency log; failure means signatures cannot be recorded or verified, blocking deployments).

The mitigations are component-specific. Neo4j: a causal cluster with three nodes across three availability zones, with daily full backups and continuous incremental backups to object storage; in a cluster-wide failure, the system degrades (the ATG is unavailable, but the data plane continues to operate, with the per-PR Conftest gate falling back to evaluating all policies rather than the touch-query subset). The OPA bundle server: deployed as a Kubernetes Deployment with 3 replicas, with the bundle cached at every OPA instance for 24 h; in a bundle server failure, OPA instances serve the cached bundle and the self-audit flags the staleness. Rekor: use the public-good Rekor (which is itself highly available) for non-air-gapped deployments; for air-gapped deployments, deploy a self-hosted Rekor cluster with the same HA

characteristics; in a Rekor failure, fall back to signature verification without log entry (with documented justification, break-glass).

10.3 Limitation: Vendor Lock-In

Each component of the RC2 architecture is open-source and replaceable, but the migration cost is real. Replacing Neo4j with Amazon Neptune requires rewriting every Cypher query (Neptune uses Gremlin and SPARQL, not Cypher). Replacing OPA with a different policy engine requires rewriting every Rego policy. Replacing Backstage with a different developer portal requires rewriting every scorecard and every plugin. The architecture is open-source, but it is not vendor-neutral in the practical sense of "easy to swap."

The mitigation is the ADR (Architecture Decision Record) discipline: every technology choice (Chapter 11, Appendix C) is documented with its trade-offs, its alternatives, and its migration cost. The ADRs make the lock-in explicit and the migration path known. The mitigation is also the interface discipline: where possible, components are accessed via standard interfaces (OpenTelemetry for observability, OSCAL for certification, JSON Schema for validation, OpenAPI for APIs), so a component swap requires changes at the interface boundary, not throughout the system. The lock-in is the cost of using best-of-breed components; the mitigation is the discipline of ADRs and standard interfaces.

10.4 Counterargument: "This Is Over-Engineered"

A common counterargument to the RC2 architecture is that it is over-engineered: 20 components, 16 detailed specifications, 5 pillars, a 7-year audit retention requirement, SLSA L3 supply-chain integrity — all for a scholarly publishing platform. The counterargument has merit: for a small, single-tenant deployment with a handful of journals and a trusted engineering team, the RC2 architecture is more than is needed. The counterargument is addressed by the scope of Opus Publica's ambition: the platform publishes across multiple journals, integrates with Crossref, ORCID, CLOCKSS, LOCKSS, Portico, DOAJ, Scopus, and Web of Science, is subject to funder-compliance and indexer-eligibility audits, and is the permanent record of scholarship. The scholarly record is the most durable artifact of human civilization; the platform that maintains it must be governed to the highest standard. The RC2 architecture is not over-engineered for the scholarly record; it is appropriately engineered.

The counterargument is also addressed by Section 10.6 (when the architecture does not apply): for small deployments, a subset of the architecture suffices, and the architecture is designed to be deployable incrementally (the RC2 Roadmap's four-phase migration). The architecture is a menu, not a fixed meal: a deployment that does not need the full menu can choose the dishes it needs. The full menu is specified for the deployment that needs the full capability, which is Opus Publica's deployment.

10.5 Counterargument: "We Could Do This with Fewer Components"

A related counterargument is that the architecture could be achieved with fewer components: a single policy engine, a single database (relational, not graph), a single dashboard, a single signing tool. The

counterargument is partially correct: the architecture could be simplified, and a simpler architecture would be easier to operate. The counterargument is also partially incorrect: the simplifications lose capability. A relational database instead of Neo4j would make traceability queries O(joins^depth) instead of O(depth), turning millisecond queries into multi-second queries and making the ATG unusable for real-time investigation. A single dashboard instead of Grafana + Backstage would force operators and developers to share a surface, optimizing for one audience at the expense of the other. A single policy engine (e.g., Gatekeeper for everything) would lose the PR-time enforcement (Conftest) and the runtime enforcement (sidecar), forcing every policy decision to deploy-time.

The counterargument is addressed by the ADR discipline: each component was chosen over its alternatives, with the trade-offs documented. The counterargument is also addressed by the deployment incrementality: a deployment can start with a subset (e.g., Conftest + GitHub Actions + a relational traceability table) and add components as the need arises. The architecture is the target state for the deployment that needs the full capability; the path to the target state is incremental, and the intermediate states are valid.

10.6 When the Architecture Does Not Apply

The RC2 architecture does not apply to every deployment. The architecture is designed for a multi-journal, multi-tenant scholarly publishing platform with external integrations and audit obligations; deployments that do not match this profile can use a subset or a different architecture entirely. The architecture does not apply in the following cases:

- **Small single-tenant deployment:** a single-journal deployment with a small editorial team and no external audit obligations can use a subset (MRC + Conftest + a relational traceability table) without the ATG, the OSCAL feed, or the self-audit engine.
- **Non-certified deployment:** a deployment that does not need continuous certification (e.g., a research prototype) can use the MRC and Conftest without the CCE, the dashboard, or the self-audit.
- **Externally-governed deployment:** a deployment that is already governed by an external system (e.g., a funder's compliance platform) can integrate with that system via OSCAL rather than deploying the full RC2 control plane.
- **Air-gapped deployment:** a deployment that cannot use external services (e.g., an air-gapped national repository) must self-host Fulcio and Rekor (documented in ADR-011) and may choose a different policy engine or graph database per ADRs.

The architecture is a target, not a mandate. Deployments that do not match the target profile are not defective; they are different. The architecture is specified for the deployment that matches the target profile, which is Opus Publica's production deployment.

Chapter 11 — Conclusion and Implications

11.1 Summary of the Architecture

This specification has detailed the RC2 Technical Architecture: a control plane (MRC, OPA, Neo4j ATG, GitHub Actions CCE, Grafana + Backstage dashboard, Self-Audit Engine) that governs a data plane (20 application subsystems, event backbone, storage, external integrations), deployed on Kubernetes with GitOps (Flux), secured with SLSA L3 supply-chain integrity (Sigstore) and a STRIDE-based threat model, observed with OpenTelemetry / Prometheus / Loki / Tempo, and continuously certified in OSCAL. The architecture operationalizes the RC2 Evolution Roadmap's five pillars as a deployable system, converting the Constitution from a document humans read into an operating system machines execute.

The architecture's distinctive contribution is the closed loop: every constitutional provision is encoded in the MRC, enforced by OPA at PR/deploy/runtime, traced in the ATG, certified in the OSCAL-AR feed, monitored on the dashboard, and audited by the Self-Audit Engine, with anomalies feeding back into the MRC as policy updates. The loop is the operational realization of continuous assurance: compliance is not a state verified at milestones but a property of the running system, verified every commit, every deploy, every night. The architecture is the implementation of the RC2 prime directive: every constitutional provision that can be evaluated by a machine IS evaluated by a machine, at the earliest possible point, with the result recorded as evidence.

11.2 Implementation Priorities (Build Order)

The architecture is built in the order specified by the RC2 Roadmap's four-phase migration: Phase 0 (Foundation: MRC repository and provision encoding), Phase 1 (Policy Encoding: Rego policies and Conftest in report-only mode), Phase 2 (Graph and Dashboard: Neo4j, Grafana, Backstage, Self-Audit), Phase 3 (Enforcement Cutover: Conftest blocking, Gatekeeper required, Production Acceptance Gate active). The build order is dependency-driven: each phase depends on the prior phase's outputs. The build order is also value-driven: each phase delivers value independently (Phase 0 delivers a single source of truth; Phase 1 delivers automated PR-time checking; Phase 2 delivers real-time visibility; Phase 3 delivers automated enforcement). The build order is specified in the Roadmap; this specification's contribution is the component-level detail that makes the build order actionable.

11.3 Implications for Team Structure (Conway's Law)

Conway's Law observes that a system's architecture mirrors the communication structure of the organization that builds it. The RC2 architecture, with its separation of control plane and data plane, implies a team structure with a corresponding separation: a Platform Engineering team (owns the control plane: MRC, OPA, Neo4j, GitHub Actions, Grafana, Backstage, Self-Audit) and Application Engineering teams (own the data plane: per-subsystem or per-subsystem-group). The separation is not optional: a

team that owns both the control plane and a data-plane subsystem has a conflict of interest (the team governs itself, which the control-plane / data-plane separation principle forbids).

The Platform Engineering team is small (3 to 5 engineers) and stable; it owns the governance machinery that every other team uses. The Application Engineering teams are organized by bounded context (e.g., a Submission team, a Publication team, a Preservation team); each team owns its subsystem end-to-end (code, tests, deployment, on-call). The teams communicate via the architecture's interfaces: an Application team consumes the Platform team's OPA bundle, ATG query API, and Backstage scorecard; the Platform team consumes the Application team's OpenTelemetry traces and audit log entries. The communication structure mirrors the architecture: a small, stable Platform team in the center, surrounded by Application teams, with well-defined interfaces between them. Conway's Law is not a constraint to resist; it is a design force to embrace.

11.4 Future Evolution (RC3 and Beyond)

The RC2 architecture is the foundation for RC3 and beyond. The RC3 horizon, as sketched in the RC2 Roadmap, includes: (1) AI-assisted policy authoring (an LLM that drafts Rego policies from natural-language provision statements, with human review); (2) predictive compliance (ML models that forecast which criteria are at risk of failing before they fail, enabling preventive remediation); (3) cross-platform constitutional federation (the OSCAL-AR feed shared, with consent, across the scholarly-infrastructure ecosystem, so a funder or indexer can verify a journal's compliance without a manual audit); (4) constitutional simulation (a tool that simulates the impact of a proposed constitutional amendment on the platform's compliance posture before the amendment is adopted).

Beyond RC3, the architecture supports multi-region deployment (the control plane's components are HA and the data plane's RDBMS supports cross-region replication), federation (multiple Opus Publica instances sharing an OSCAL-AR feed and a constitutional core), and AI-native governance (the MRC's provisions as a knowledge base for an LLM that answers governance questions in natural language). These are speculative; they depend on RC2's success. But RC2 is the foundation that makes them conceivable: without a machine-readable constitution, there is nothing for an LLM to author; without continuous certification, there is no data for predictive models; without OSCAL, there is no format for federation. RC2 is the precondition for the future.

Appendices

Appendix A: Glossary of Architectural Terms

The following terms are used throughout this specification. Definitions are normative for this document.

Term	Definition
ADR	Architecture Decision Record; a document that captures a technology or design choice, its alternatives, its trade-offs, and its migration cost. See Appendix C for the ADR template.
ATG	Automated Traceability Graph; the live property graph (Neo4j) of engineering artifacts and traceability edges.
Bundle (OPA)	A signed tarball of Rego policies and data, served by the bundle server and fetched by OPA instances.
CCE	Continuous Certification Engine; the integrated GitHub Actions + Conftest + OSCAL emitter that evaluates every certification criterion continuously.
Conftest	An OPA subproject for testing structured configuration data against Rego policies; the PR-time enforcement point.
Control Plane	The set of RC2 components that govern the platform (MRC, OPA, Neo4j, GitHub Actions, Grafana, Backstage, Self-Audit).
Data Plane	The set of RC2 components that produce and consume user-visible data (the 20 application subsystems, the event backbone, storage, external integrations).
Gatekeeper	OPA Gatekeeper; the Kubernetes admission webhook that enforces Rego policies at deploy time.
GitOps	The operational model in which Git is the source of truth for system state and a controller (Flux) reconciles live state to declared state.
ISCM	Information Security Continuous Monitoring; NIST SP 800-137's 6-phase lifecycle (Define, Establish, Implement, Analyze, Report, Respond, Review).
MRC	Machine-Readable Constitution; the versioned YAML/JSON encoding of every constitutional provision, owned in Git.

OPA	Open Policy Agent; the CNCF Graduated policy engine that evaluates Rego policies.
OSCAL	Open Security Controls Assessment Language; a NIST-led machine-readable format for security control documentation, including Assessment Results.
OSCAL-AR	OSCAL Assessment Results; the machine-readable certification record produced by the CCE.
Outbox Pattern	A pattern in which a subsystem writes to its database and to an outbox table in the same transaction; a separate process publishes the outbox to the event backbone, guaranteeing atomicity.
Policy-as-Code	The discipline of expressing governance policies as versioned, testable, reviewable code (Rego).
Rego	OPA's declarative policy language.
SAP / SAE	Self-Auditing Platform / Self-Audit Engine; the continuous audit component that realizes continuous assurance.
Shift-Left	The principle of moving verification as early as possible in the engineering lifecycle.
SLSA	Supply-chain Levels for Software Artifacts; the OpenSSF framework defining supply-chain integrity levels (L1-L4).
STRIDE	Microsoft's threat-modeling framework: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.

Appendix B: Component Dependency Matrix

The following matrix shows the dependencies between the RC2 components. A row's component depends on the column's component if the cell is 'D' (depends); a reverse 'R' indicates the reverse dependency. The matrix is useful for impact analysis: a change to a column component requires reviewing every row component marked 'D'.

Component	MRC	OPA	Neo4j	GH Actions	Grafana	Backstage	Sigstore	OSCAL	Self-Audit
MRC Repository	—			D			D	D	

Document Generators	D			D			D	D	
OPA Policy Engine	D	—					D		
Conftest Runner	D	D	D	D			D	D	
GitHub Actions CI/CD	D			—			D	D	D
Gatekeeper	D	D							
Flux GitOps							D		
Neo4j ATG	D		—						
Graph Builders	D		D	D					
OTel Collector					D				
Prometheus + Grafana			D		—			D	D
Backstage Portal	D		D			—		D	
Sigstore				D			—		
OSCAL Emitter	D			D			D	—	
Self-Audit Engine	D	D	D	D	D	D		D	—
Anomaly Detection		D	D		D				D

Reading the matrix: the Self-Audit Engine (row) depends on the MRC, OPA, Neo4j, GitHub Actions, Grafana, Backstage, and OSCAL (columns marked 'D'), because the Self-Audit queries the ATG, the OSCAL-AR feed, and the Grafana summary, and emits findings to PagerDuty and Jira. A change to any of those components requires reviewing the Self-Audit Engine for compatibility. The matrix is symmetric where dependencies are bidirectional (e.g., the ATG depends on the MRC for the Constitution Builder; the Self-Audit depends on the ATG for integrity queries).

Appendix C: ADR Template

Every architectural decision in RC2 is recorded as an Architecture Decision Record (ADR). The ADR template below is normative: an ADR that does not follow this template is defective. ADRs are stored in the MRC repository (adr/ directory) and are linked from the ATG as ADR nodes.

```
# ADR-NNNN: <Decision Title>

- **Status:** Proposed | Accepted | Deprecated | Superseded by ADR-MMMM
- **Date:** YYYY-MM-DD
- **Deciders:** <names>
- **Consulted:** <names>
- **Informed:** <names>

## Context
<The problem, the forces, the constraints. What is being decided and why now?>

## Decision
<The choice that is being made. State it in a single sentence first, then expand.>

## Alternatives Considered
### Alternative A: <name>
- **Pros:** <list>
- **Cons:** <list>
- **Migration cost:** <estimate>
### Alternative B: <name>
- **Pros:** <list>
- **Cons:** <list>
- **Migration cost:** <estimate>

## Trade-Offs
<What we gain; what we lose; why the gain is worth the loss.>

## Consequences
- **Positive:** <list>
- **Negative:** <list>
- **Neutral:** <list>

## Compliance
<Which constitutional provisions does this decision affect? Which ADRs does it supersede?
Which ATG nodes does it touch?>

## References
<URLs, documents, prior ADRs, standards.>
```

The ADRs triggered by this specification include: ADR-001 (MRC encoding: YAML + JSON Schema 2020-12 + CUE for authoring), ADR-002 (OPA as policy engine), ADR-003 (Neo4j as ATG), ADR-004 (GitHub Actions

as CI/CD substrate), ADR-005 (Flux as GitOps controller), ADR-006 (Backstage as developer portal), ADR-007 (Grafana as operator dashboard), ADR-008 (Sigstore as supply-chain signing), ADR-009 (OSCAL as certification format), ADR-010 (OpenTelemetry as observability framework), ADR-011 (Self-hosted Sigstore for air-gapped deployments). Each ADR follows the template above and is reviewed by the Architecture Review Board (the CSA and the Principal Architects) before acceptance.

Appendix D: References

The following references are the canonical authorities cited in this specification. All URLs were verified during authoring (August 2026).

D.1 Standards and Specifications

NIST SP 800-137. Information Security Continuous Monitoring (ISCM). https://csrc.nist.gov/pubs/sp/800/137/final

NIST SP 800-204C. Implementation of DevSecOps for Microservices with Service Mesh (policy-as-code). https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204c.pdf

NIST SP 800-92. Guide to Computer Security Log Management. https://csrc.nist.gov/pubs/sp/800/92/final

NIST OSCAL. Open Security Controls Assessment Language. https://pages.nist.gov/OSCAL/

RFC 2119. Key words for use in RFCs to indicate requirement levels. https://datatracker.ietf.org/doc/html/rfc2119

RFC 6902. JSON Patch. https://www.rfc-editor.org/info/rfc6902

RFC 8446. TLS 1.3. https://datatracker.ietf.org/doc/html/rfc8446

JSON Schema Draft 2020-12. JSON Schema specification. https://json-schema.org/draft/2020-12

YAML 1.2.2. YAML Ain't Markup Language specification. https://yaml.org/spec/1.2.2/

OpenAPI 3.1. OpenAPI Specification (aligned with JSON Schema 2020-12). https://swagger.io/specification/

Conventional Commits 1.0.0. Commit message specification. https://www.conventionalcommits.org/en/v1.0.0

Semantic Versioning 2.0.0. Version number specification. https://semver.org/spec/v2.0.0.html

C4 Model. Simon Brown's Context, Container, Component, Code model. https://c4model.com/

SLSA v1.0. Supply-chain Levels for Software Artifacts. https://slsa.dev/spec/v1.0/

CycloneDX 1.6. OWASP-originated SBOM standard (ECMA-424). https://cyclonedx.org/specification/overview/

ISO/IEC 5962:2021. SPDX SBOM standard. https://www.iso.org/standard/73267.html

CNCF OpenGitOps Principles. GitOps principles v1.0.0. https://opengitops.dev/

D.2 Technologies and Projects

Open Policy Agent (OPA). CNCF Graduated policy engine; Rego policy language. https://www.openpolicyagent.org/

OPA Gatekeeper. Kubernetes admission webhook for OPA. https://open-policy-agent.github.io/gatekeeper/

Conftest. OPA subproject for testing structured configuration data. https://www.conftest.dev/

CUE. Configure, Unify, Execute; data-validation language. https://cuelang.org/

Backstage. Spotify open-source developer portal framework (CNCF). https://backstage.io/

Neo4j. Market-leading property-graph database; Cypher query language. https://neo4j.com/

Amazon Neptune. AWS-managed graph database (alternative to Neo4j). https://aws.amazon.com/neptune/

Memgraph. In-memory graph database (alternative to Neo4j). https://memgraph.com/

OpenTelemetry. CNCF vendor-neutral observability framework. https://opentelemetry.io/

Prometheus. CNCF Graduated monitoring and alerting toolkit. https://prometheus.io/

Grafana. Open-source visualization and dashboarding platform. https://grafana.com/

Loki. Grafana Labs horizontal log aggregation system. https://grafana.com/oss/loki/

Tempo. Grafana Labs distributed tracing backend. https://grafana.com/oss/tempo/

Flux. CNCF Graduated GitOps controller. https://fluxcd.io/

Argo CD. CNCF Graduated GitOps controller (alternative to Flux). https://argoproj.github.io/argo-cd/

Sigstore. OpenSSF code-signing stack (Cosign, Rekor, Fulcio). https://www.sigstore.dev/

Cosign. Sigstore's container signing CLI. https://github.com/sigstore/cosign

Rekor. Sigstore's transparency log. https://github.com/sigstore/rekor

Fulcio. Sigstore's short-lived OIDC-bound certificate authority. https://github.com/sigstore/fulcio

slsa-github-generator. SLSA L3 provenance generator for GitHub Actions. https://github.com/slsa-framework/slsa-github-generator

Syft. SBOM generation tool (Anchore). https://github.com/anchore/syft

driftctl. Infrastructure drift detection (Snyk). https://github.com/snyk/driftctl

HashiCorp Vault. Secret management. https://developer.hashicorp.com/vault

External Secrets Operator. Kubernetes operator for syncing external secrets. https://external-secrets.io/

GitHub Actions. GitHub CI/CD platform. https://docs.github.com/en/actions

actions-runner-controller. Kubernetes-based self-hosted GitHub runners. https://github.com/actions/actions-runner-controller

Debezium. Change Data Capture for the outbox pattern. https://debezium.io/

Apache Kafka. Distributed event streaming platform. https://kafka.apache.org/

Redpanda. Kafka-compatible streaming platform (alternative). https://redpanda.com/

PostgreSQL. Open-source relational database. https://www.postgresql.org/

MinIO. S3-compatible object storage (self-hosted alternative). https://min.io/

Confluent Schema Registry. Schema evolution and compatibility types. https://docs.confluent.io/platform/current/schema-registry/index.html

D.3 Opus Publica Internal Documents

RC2 Evolution Roadmap v1.0. Strategic document this specification operationalizes; specifies the five pillars and the migration path. (internal: /download/Opus_Publica_RC2_Evolution_Roadmap_v1.0.docx)

RC1 Engineering Constitution Vol I. 25-chapter specification of what the platform is; provisions enforced by RC2. (internal: /download/Opus_Publica_RC1_Engineering_Constitution_v1.0.docx)

RC1 Engineering Constitution Vol II. 11-chapter specification of invariants, contracts, events, thresholds. (internal: /download/Opus_Publica_RC1_Engineering_Constitution_Vol_II_v1.0.docx)

RC1 Engineering Playbook. 20-chapter operational procedures that RC2 automates. (internal: /download/Opus_Publica_RC1_Engineering_Playbook_v1.0.docx)

RC1 Certification Checklist. 326-page objective pass/fail criteria encoded as Rego policies in RC2. (internal: /download/Opus_Publica_RC1_Certification_Checklist_v1.0.docx)

RC1 Constitutional Traceability Matrix. 191-page hand-maintained matrix replaced by the live ATG in RC2. (internal: /download/Opus_Publica_RC1_Constitutional_Traceability_Matrix_v1.0.docx)

D.4 Industry and Conceptual References

Google SRE Book. Site Reliability Engineering (free online). https://sre.google/sre-book/

Google SRE Workbook. Implementing SLOs; alerting on SLOs. https://sre.google/workbook/

ISACA. Continuous assurance in cloud compliance. https://www.isaca.org/resources/news-and-trends/isaca-now-blog/2025/cloud-compliance-continuous-assurance-harnessing-ai-rpa-and-cspm-for-a-new-era-of-trust

Microsoft STRIDE. Threat-modeling framework. https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats#stride-model

Conway's Law. Melvin Conway, 1968; organizations design systems that mirror their communication structure. https://en.wikipedia.org/wiki/Conway%27s_law

OWASP ASVS 4.0. Application Security Verification Standard. https://owasp.org/www-project-application-security-verification-standard/

— Issued under the authority of the Chief Systems Architect, Opus Publica — RC2 Technical Architecture Specification —